



Hochschule für Technik,
Wirtschaft und Kultur Leipzig

Bachelor Thesis

Entwicklung einer Mobilen Applikation zur
effizienten Stimmung einer
Floyd-Rose-Gitarre

Name: Raphael Schütz

Matrikelnummer: 82832

Fachsemester: 8

Erstprüfer: Prof. Konrad Schöbel

Zweitprüfer: Prof. Ulf Schemmert

Abgabe: 09.06.2026

Inhaltsverzeichnis

1	Motivation	5
2	Grundlagen (Physik der Gitarre)	5
2.1	Experiment: Elastizität von Gitarrensaiten	6
2.2	Physikalisches Modell	7
2.3	Experiment: Nachweis Linearität	12
2.3.1	Vorgehensweise	12
2.3.2	Ergebnisse	13
2.3.2.1	Relative Visualisierung der Frequenzänderungen	13
2.3.2.2	Korrelationskoeffizienten und Fehler	14
2.3.3	Diskussion der Ergebnisse	14
2.4	Mathematische Lösung	14
2.5	Praktische Umwandlung	16
3	Ablauf eines Stimmvorgangs	16
3.1	Initiierung – Bestimmung der Verstimmungsmatrix	16
3.2	Stimmvorgang	17
3.3	Überprüfung	17
4	Grundlegende Verfahren	17
4.1	Bestimmung der Fundamentalfrequenz	17
4.1.1	Autokorrelation	17
4.1.2	YIN-Algorithmus	18
4.1.3	Fourier- und Cepstrum-Analyse	18
4.1.4	Moderne Deep-Learning-Ansätze	18
4.1.5	Auswahl des Verfahrens	18
4.2	Signal-Filter	18
4.2.1	Bandpassfilterung durch Parameteranpassung	18
4.2.2	Gleitender Mittelwert für Streaming-Messdaten	19
4.2.3	Amplitudenschwelle	19
5	Software Entwicklung/Implementierung	19
5.1	Requirements Engineering	19
5.1.1	Projektvision	19
5.1.2	Ziel- und Benutzergruppe	19
5.1.3	Systemkontext	20
5.1.4	Anwendungskontext	20
5.1.5	Personas	20
5.1.5.1	Emil – Gitarren-Einsteiger	20
5.1.5.2	Matilda – Professionelle Gitarristin	21
5.1.5.3	Jonas – Gitarrentechniker	21
5.1.5.4	Hanna – Home-Producerin	21
5.1.6	Szenarien	22
5.1.6.1	Szenario 1 – Emils erster Versuch	22
5.1.6.2	Szenario 2 – Matilda auf einer Jam-Session	22
5.1.7	Anforderungen	23
5.1.7.1	Funktionale Benutzeranforderungen	23
5.1.7.2	Funktionale Systemanforderungen	24
5.1.7.3	Nichtfunktionale Anforderungen	26
5.1.7.3.1	Messgenauigkeit	26
5.1.7.3.2	Latenz	26

5.1.7.3.3	Robustheit	26
5.1.7.3.4	Bedienbarkeit	27
5.1.7.3.5	Kompatibilität	27
5.1.7.3.6	Datenschutz und Betrieb	27
5.1.7.4	Ausgeschlossene Anforderungen	27
5.1.7.5	Hinweis	28
5.1.8	Entwicklungsparadigma	28
5.2	Konzeption und Design	29
5.2.1	Informationsarchitektur	29
5.2.2	Interaktionsdesign	29
5.2.2.1	Navigation	30
5.2.2.2	Kalibrierung	30
5.2.2.3	Stimmen	31
5.2.3	Visuelles Konzept	31
5.2.4	Prototypen	31
5.2.5	Wireframes	32
5.2.6	Design	34
5.2.6.1	Onboarding	34
5.2.6.2	Kalibrierung	35
5.2.6.3	Stimmvorgang	36
5.3	Architektur	37
5.3.1	Audio Stream Provider	37
5.3.2	Volume Stream Provider	37
5.3.3	Frequency Stream Provider	38
5.3.4	Smoothed Frequency Provider	38
5.3.5	Volume Threshold Provider	38
5.3.6	Detected Frequency Provider	38
5.3.7	String Measure State Provider	38
5.3.8	Guitar State Provider	38
5.3.9	Calibration State Provider	38
5.3.10	Guitars- und Selected-Guitar-Provider	38
5.3.11	Tunings- und Selected-Tuning-Provider	39
5.4	Implementierung	39
5.4.1	Auswahl des Cross-Platform-Frameworks	39
5.4.1.1	Umsetzungsgeschwindigkeit komplexer Anforderungen	39
5.4.1.2	Einstiegshürde	39
5.4.1.3	Codestabilität	40
5.4.1.4	UI-Komponenten mit Material 3	40
5.4.1.5	Entscheidung	40
5.4.2	Abhängigkeiten	40
5.4.3	Quellcode	41
5.4.4	Kalibrierungslogik	41
5.4.4.1	Fortschrittsberechnung	42
5.4.4.2	Navigationslogik	42
5.4.5	Stimmlogik	44
5.4.5.1	Fortschrittsberechnung	44
5.4.5.2	Navigationslogik	45
5.5	Tests während der Entwicklung	46
5.5.1	Manuelle Tests	46

5.5.2	Unit-Tests	46
5.5.3	Nicht getestet	46
6	Evaluation	46
6.1	Funktionsfähigkeit des Algorithmus	46
6.2	Erfüllung der Requirements aus SWE	47
6.3	Nutzertests	48
6.3.1	Nutzer 0	48
6.3.2	Nutzer 1	49
6.3.3	Nutzer 2	49
6.3.4	Nutzer 3	49
6.3.5	Nutzer 4	49
6.3.6	Nutzer 5	50
7	Ausblick	50
7.1	Optimierung der Fundamentalfrequenzschätzung	50
7.2	Umsetzung und Testen der verbleibenden Anforderungen	50
7.3	Gleichzeitiges Messen aller Saiten	50
7.4	Automatische Saitenerkennung beim Stimmen	50
7.5	Automatisches Fortfahren beim Stimmen	51
7.6	Automatische Kalibrierung der Mikrofonempfindlichkeit	51
7.7	Mehr Konfigurationsfreiraum	51
7.7.1	Änderbare Referenzstimmung	51
7.7.2	Unterstützung beliebig vieler Saiten	51
7.8	Implementierung als DAW-Plugin	51
7.9	Veröffentlichung und Erweiterung auf weitere Brückensysteme	51
	Bibliografie	57

1 Motivation

Es gibt ein Problem beim Stimmen von Floyd-Rose-Gitarren. Bei diesen Gitarren wird eine Saite zwischen dem Gitarrensattel und einer bis zu einem gewissen Grad rotierbaren Brücke gespannt. An der Brücke halten unterhalb des Drehpunkts Federn dagegen, wenn man Saiten einspannt. Die Brücke wird gemeinhin als „Floating Bridge“ bezeichnet, weil sie nicht wie herkömmliche Tremolos am Gitarrenkörper aufliegt, sondern zusätzlichen Rotationsspielraum in Richtung des Gitarrenkörpers hat.



Abbildung 1: Floyd-Rose-Tremolo Bild

Das Floyd-Rose-Tremolo hat einen Hebel, den man ziehen oder drücken kann. Beim Musizieren ändert das den Ton. Auch wenn diese Architektur beliebt ist, weil sie neue Klänge ermöglicht, erschwert sie das Stimmen der Gitarre erheblich. Beim Stimmen erhöht oder verringert man die Spannung einer Saite. Aber das führt dazu, dass die anderen Saiten verstimmt werden. Es gibt Erfahrungsberichte und Aufzeichnungen, wie man eine solche Gitarre effizient stimmen kann. Dabei dauerte das Stimmen 8 Minuten [1]. In Foren sprachen Nutzer von einer Stimmzeit von bis zu 20-30 Minuten, je nachdem, wie sauber und wie viele Saiten sie stimmen mussten [2]. Ziel der Arbeit ist eine App zu entwickeln, die diesen Stimmvorgang beschleunigt.

2 Grundlagen (Physik der Gitarre)

Die Gitarre spannt 6 Saiten zwischen Brücke und Sattel. Die Saiten schwingen in einer bestimmten Frequenz. Beim Stimmen wickelt man die Saite um den Stimmwirbel, sodass sich Spannung und Frequenz ändert. Dass Saiten elastisch sind, wird im folgenden Experiment gezeigt:



Abbildung 2: Gitarre Stimmwirbel

2.1 Experiment: Elastizität von Gitarrensaiten

Die Ergebnisse des Experiments stammen aus der Arbeit des Autors, welche im Rahmen des Moduls „Projekt 3“ aus dem Telekommunikationsinformatik-Studium an der HTWK-Leipzig [3] angefertigt wurden.

In der folgenden Tabelle sind Bilder, die die elastische Dehnung der Saite zeigen:

Bund	Pacifica - Ruhe	Pacifica - Spannung
1	 Abbildung 3: Bund 1 - Ruhe	 Abbildung 4: Bund 1 - Spannung
2	 Abbildung 5: Bund 2 - Ruhe	 Abbildung 6: Bund 2 - Spannung
4	 Abbildung 7: Bund 4 - Ruhe	 Abbildung 8: Bund 4 - Spannung
6	 Abbildung 9: Bund 6 - Ruhe	 Abbildung 10: Bund 6 - Spannung
8	 	

Bund	Pacifica - Ruhe	Pacifica - Spannung
	Abbildung 11: Bund 8 - Ruhe	Abbildung 12: Bund 8 - Spannung
12	 Abbildung 13: Bund 12 - Ruhe	 Abbildung 14: Bund 12 - Spannung
16	 Abbildung 15: Bund 16 - Ruhe	 Abbildung 16: Bund 16 - Spannung
22	 Abbildung 17: Bund 22 - Ruhe	 Abbildung 18: Bund 22 - Spannung

Tabelle 1: Elastizität von Gitarrensaiten

Die Markierungen, die sich näher am Sattel befanden, legten eine deutlich größere Strecke zurück als jene in unmittelbarer Nähe der Brücke. Die beobachtete Verschiebung nahm kontinuierlich vom Sattel in Richtung Brücke ab.

Die Kontrollmarkierungen auf den übrigen Saiten zeigten dagegen keine oder lediglich eine kaum wahrnehmbare Bewegung. Dies spricht dafür, dass die beobachtete Verschiebung nicht durch ein Verformen des Instruments verursacht wurde, sondern auf eine tatsächliche Längenänderung der gespannten Saite zurückzuführen ist. Als die Saite wieder entspannt wurde, waren die Markierungen wieder an ihrer Ausgangsposition. Die Schwingungsfrequenz der Saite war auch wieder dieselbe wie zu Beginn.

Die Beobachtungen belegen das elastische Verhalten von Gitarrensaiten. Wird die Spannung durch Aufwickeln am Stimmwirbel erhöht, verschieben sich die aufgeklebten Markierungen entlang der Saite in unterschiedlichem Ausmaß. Markierungen in der Nähe der Brücke, die als nahezu fixer Punkt wirkt, erfahren nur eine sehr geringe Verschiebung, während weiter entfernte Markierungen deutlich stärker wandern.

2.2 Physikalisches Modell

Im Folgenden wird ein physikalisches Modell der Gitarre beschrieben, um zu verstehen warum die Floyd-Rose-Gitarre so schwierig zu stimmen ist.

vektor $\vec{f} = \begin{pmatrix} f_1 \\ \vdots \\ f_i \\ \vdots \\ f_6 \end{pmatrix}$ abbildet $\overrightarrow{\Delta L} \rightarrow \vec{f}$, wobei jede Komponente zu einer Saite gehört. Beim Stimmen muss $\overrightarrow{\Delta L}$ so gewählt werden, dass genau die gewünschten Frequenzen erreicht werden. Das Ziel ist die Funktion $f(\overrightarrow{\Delta L})$ zu bestimmen. Der Zusammenhang zwischen effektiver Saitenlänge $L_{S,i}$, Zugkraft $F_{S,i}$, linearer Massendichte μ_i und Frequenz f_i wird durch das Mersennesche Gesetz beschrieben [4]:

Zunächst wird die Saitenkraft $F_{S,i}$ als Funktion der Aufwickelstrecken $\overrightarrow{\Delta L}$ bestimmt. Die Kraft die auf die Saite wirkt, wird durch das Hooksche Gesetz beschrieben [5]:

$L_{0S,i}$ beschreibt die unbelastete Saitenlänge im Abschnitt zwischen Sattel und Brücke. Diese Länge wird durch die Aufwickelstrecke ΔL_i beeinflusst. $L'_{0S,i}$ sei die initiale unbelastete Saitenlänge.

8

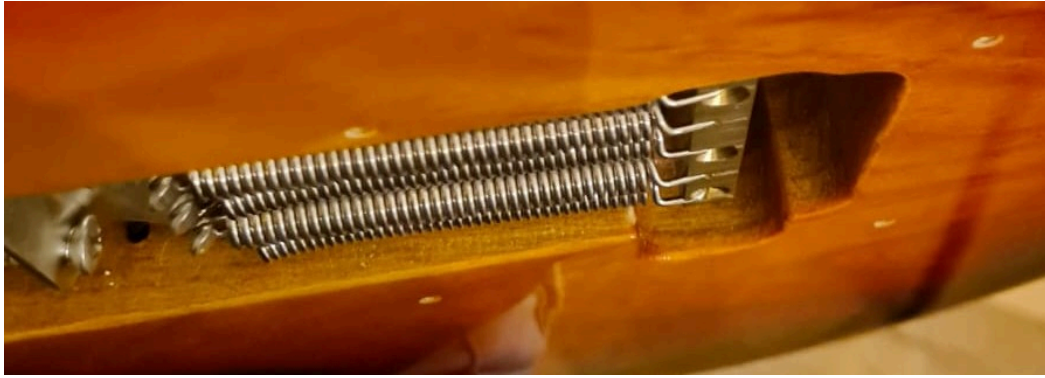


Abbildung 20: Tremolofedern

In Abbildung 1, Abbildung 19 und Abbildung 20 ist zu sehen, wie die Brücke die Tremolofedern und die Saiten über einen Drehmoment koppelt. Die Tremolofedern dienen unterhalb der Brücke als Gegenkraft zu der Saitenspannung.

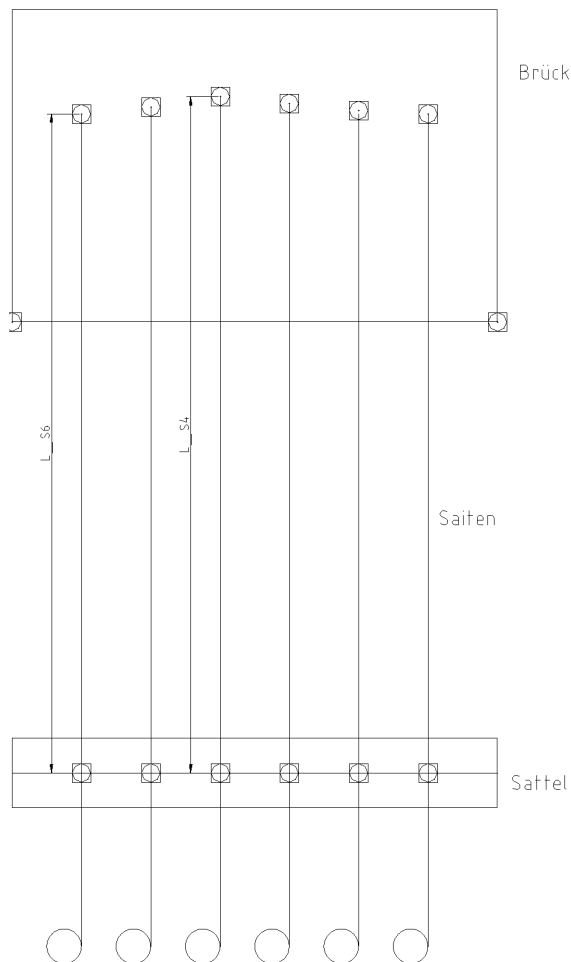


Abbildung 21: Floyd-Rose-Modell Draufsicht



Abbildung 22: Floyd-Rose Draufsicht

In der Realität hat jede Saite ihre eigene Saitenlänge, wie in Abbildung 21 und Abbildung 22 zu sehen ist. Sie variieren zwar nur minimal haben aber einen Einfluss auf die rotatorische Projektion der Kräfte.

Die Brücke wird als starrer, gewinkelter Hebel betrachtet, siehe Abbildung 19. Die Drehachse liege im Koordinatenursprung. Die Vektoren $\vec{h}_F$ (Hebelarm der Feder) und $\vec{h}_{S,i}$ (Hebelarm der Saite i) schließen konstruktionsbedingt einen konstanten Winkel α_i ein. Die Beträge h_F und $h_{S,i}$ sind systemspezifische

Konstanten. Jede Saite erhält ihren eigenen Hebelarm $\overrightarrow{h_{S,i}}$, um den Aufbau wie in Abbildung 22 und Abbildung 21 korrekt zu modellieren. Die Tremolofedern erhalten in diesem Modell einen gemeinsamen Hebelarm $\overrightarrow{h_{\hat{F}}}$.

$$\overrightarrow{h_{\hat{F}}}(\beta) = h_{\hat{F}} \begin{pmatrix} \cos(\beta) \\ \sin(\beta) \end{pmatrix} \quad (4)$$

$$\overrightarrow{h_{S,i}}(\beta) = h_{S,i} \begin{pmatrix} \cos(\beta + \alpha_i) \\ \sin(\beta + \alpha_i) \end{pmatrix} \quad (5)$$

Sei $\overrightarrow{P_S}$ die konstante Position des Sattels. Die effektive Saitenlänge und Tremolofederlänge ergeben sich zu

$$L_{S,i}(\beta) = |\overrightarrow{h_{S,i}}(\beta) - \overrightarrow{P_S}| \quad (6)$$

$$L_{\hat{F}}(\beta) = |\overrightarrow{h_{\hat{F}}}(\beta) - \overrightarrow{P_{\hat{F}}}| \quad (7)$$

Nun soll die Variable β bestimmt werden, die sich aus dem Kräftegleichgewicht und der darauffolgenden Hebelposition ergibt. Nach den Gesetzen der Statik trägt ausschließlich der zur jeweiligen Hebelarmrichtung orthogonale Kraftanteil zum Drehmoment bei [6]. Im stationären Gleichgewicht gilt das Drehmomentgleichgewicht:

$$\sum_{i=1}^6 F_{S,i,\perp h_{S,i}} \cdot h_{S,i} = F_{\hat{F},\perp h_{\hat{F}}} \cdot h_{\hat{F}} \quad (8)$$

Dabei bezeichnen $F_{S,i,\perp h_{S,i}}$ und $F_{\hat{F},\perp h_{\hat{F}}}$ jeweils die Anteile der Kräfte $\overrightarrow{F_{S,i}}$ und $\overrightarrow{F_{\hat{F}}}$, die orthogonal zu den Hebelarmen $\overrightarrow{h_{S,i}}$ und $\overrightarrow{h_{\hat{F}}}$ wirken. Auf der linken Seite von Gleichung 8 müssen die Kräfte der 6 Saiten aufaddiert werden, da sich die Kräfte parallelgeschalteter Federn addieren [5].

Zunächst wird der Richtungsvektor von $F_{S,i}$, $h_{S,i}$, $F_{\hat{F}}$ und $h_{\hat{F}}$ normiert, wobei $P_{\hat{F}}$ der Punkt ist, an dem die Tremolofeder an der Gitarre befestigt ist.

$$\overrightarrow{e_{F_{S,i}}} = \frac{\overrightarrow{P_S} - \overrightarrow{h_{S,i}}}{|\overrightarrow{P_S} - \overrightarrow{h_{S,i}}|} \quad (9)$$

$$\overrightarrow{e_{h_{S,i}}} = \begin{pmatrix} \cos(\beta + \alpha_i) \\ \sin(\beta + \alpha_i) \end{pmatrix} \quad (10)$$

$$\overrightarrow{e_{F_{\hat{F}}}} = \frac{\overrightarrow{P_{\hat{F}}} - \overrightarrow{h_{\hat{F}}}}{|\overrightarrow{P_{\hat{F}}} - \overrightarrow{h_{\hat{F}}}|} \quad (11)$$

$$\overrightarrow{e_{h_{\hat{F}}}} = \begin{pmatrix} \cos(\beta) \\ \sin(\beta) \end{pmatrix} \quad (12)$$

Aus der orthogonalen Projektion eines Vektors $\vec{a}$ bezüglich eines Vektors $\vec{b}$ folgt [7]:

$$F_{S,i,\perp h_{S,i}} = F_{S,i} \cdot \sqrt{1 - (\overrightarrow{e_{F_{S,i}}} \cdot \overrightarrow{e_{h_{S,i}}})^2} \quad (13)$$

$$= F_{S,i} \cdot \sin(\angle(\overrightarrow{e_{F_{S,i}}}, \overrightarrow{e_{h_{S,i}}})) \quad (14)$$

Analog ergibt sich für die Tremolofeder:

$$F_{\hat{F}, \perp h_{\hat{F}}} = F_{\hat{F}} \cdot \sqrt{1 - (\overrightarrow{e_{F_{\hat{F}}}} \cdot \overrightarrow{e_{h_{\hat{F}}}})^2} \quad (15)$$

$$= F_{\hat{F}} \cdot \sin(\angle(\overrightarrow{e_{F_{\hat{F}}}}, \overrightarrow{e_{h_{\hat{F}}}})) \quad (16)$$

Das Kräftegleichgewicht lässt sich damit schreiben als:

$$\sum_{i=1}^6 F_{S,i} \sin(\angle(\overrightarrow{e_{F_{S,i}}}, \overrightarrow{e_{h_{S,i}}})) \cdot h_{S,i} = F_{\hat{F}} \sin(\angle(\overrightarrow{e_{F_{\hat{F}}}}, \overrightarrow{e_{h_{\hat{F}}}})) \cdot h_{\hat{F}} \quad (17)$$

Der nächste Schritt wäre, diesen Ausdruck nach $\beta(\overrightarrow{\Delta L})$ umzustellen, um die Hebelposition zu bestimmen. Allerdings ist dies nicht analytisch möglich, da β in den Sinusfunktionen und den Hebelarmvektoren auf beiden Seiten der Gleichung vorkommt. Es liegt ein nichtlineares Gleichungssystem vor, das numerisch gelöst werden muss.

Bringt man sie in die Form einer Nullstellengleichung, erhält man

$$0 = g(\beta; \overrightarrow{\Delta L}) = \sum_{i=1}^6 F_{S,i} \sin(\angle(\overrightarrow{e_{F_{S,i}}}, \overrightarrow{e_{h_{S,i}}})) \cdot h_{S,i} - F_{\hat{F}} \sin(\angle(\overrightarrow{e_{F_{\hat{F}}}}, \overrightarrow{e_{h_{\hat{F}}}})) \cdot h_{\hat{F}} \quad (18)$$

Damit liegt ein eindimensionales nichtlineares Optimierungs- bzw. Nullstellenproblem vor, mit dem sich der Rotationswinkel β numerisch bestimmen lässt.

Aus dem so berechneten Winkel ergeben sich transitiv die abhängigen Größen $h_{S,i}(\beta)$, $L_{S,i}(\beta)$ und $F_{S,i}(\overrightarrow{\Delta L})$.

μ_i ist von der Aufwickelstrecke $\overrightarrow{\Delta L}$ abhängig. Im Allgemeinen gilt für Saite i :

$$\mu_i = \frac{m_i}{L_{S,i,\text{Total}}} \quad (19)$$

Dabei bezeichnet m_i die Gesamtmasse und $L_{S,i,\text{Total}}$ die Gesamtlänge der Saite i . Die Gesamtlänge setzt sich zusammen aus der effektiven Saitenlänge $L_{S,i}$ und der Teil der Saite der hinter dem Sattel liegt, wie in Abbildung 2 und Abbildung 21 zu sehen ist.

Diese Strecke sei $L_{M,i} = L_{0M,i} + \Delta L_i$.

$L_{0M,i}$ beschreibt dabei die konstante Ausgangslage.

$$L_{S,i,\text{Total}}(\overrightarrow{\Delta L}) = L_{S,i}(\overrightarrow{\Delta L}) + L_{0M,i} + \Delta L_i \quad (20)$$

Beim Aufwickeln der Saite erhöht sich die Strecke hinter dem Sattel um ΔL_i . Die zusätzliche Strecke, die durch die Dehnung entsteht steckt in $\Delta L_{S,i}(\overrightarrow{\Delta L})$.

Die lineare Massendichte ergibt sich somit zu:

$$\mu_i(\overrightarrow{\Delta L}) = \frac{m_i}{L_{S,i}(\overrightarrow{\Delta L}) + L_{0M,i} + \Delta L_i} \quad (21)$$

Darauf aufbauend lässt sich eine Abbildung definieren, die die Aufwickelstrecken jeder Saite auf einen Frequenzvektor abbildet:

$$f_i(\overrightarrow{\Delta L}) = \frac{1}{2 \cdot L_{S,i}(\overrightarrow{\Delta L})} \sqrt{\frac{F_{S,i}(\overrightarrow{\Delta L})}{\mu_i(\overrightarrow{\Delta L})}} \quad (22)$$

Es wird ersichtlich, dass die Aufwickelstrecken der Saiten die Frequenzen aller Saiten beeinflussen. Das erklärt, warum das Stimmen einer Floyd-Rose-Gitarre so schwierig ist.

Beim Stimmen werden die Aufwickelstrecken nur in kleinen Schritten verändert. In diesem Fall verhält sich das System näherungsweise linear. Da das System physikalisch ist, können wir das System als stetig betrachten.

2.3 Experiment: Nachweis Linearität

Diese Linearität wurde bereits experimentell in der *Projekt 3* Arbeit des Autors [3] überprüft. Dabei wurde jedoch nicht der lineare Zusammenhang zwischen der Aufwickelstrecke $\overrightarrow{\Delta L}$ und den Frequenzen $\vec{f}$ betrachtet. Stattdessen analysierte die Untersuchung den Einfluss der Frequenzänderung einer Saite auf die Frequenzen der übrigen Saiten. Dieses Vorgehen ist insbesondere im Hinblick auf die spätere praktische Anwendung von höherer Relevanz.

2.3.1 Vorgehensweise

Zunächst wurde jede Saite in eine Ausgangsposition gebracht. Die Ausgangsfrequenzen der Saiten wurden in Hertz gemessen. Anschließend wurde jeweils eine Saite um ein beliebiges Δ (in Hertz) verstimmt. Dieses Δ wurde so gewählt, dass die Verstimmung deutlich hörbar ist. Für jeden Schritt wurde die Frequenz aller anderen Saiten gemessen.

Nr.	Saite	Frequenz
1	E2 = E-Saite	82.41 Hz
2	A2 = A-Saite	110 Hz
3	D3 = D-Saite	146.83 Hz
4	G3 = G-Saite	196 Hz
5	B3 = B-Saite	246.94 Hz
6	E4 = hohe E-Saite	329.63 Hz

Tabelle 2: Saitennamen mit Frequenzen

2.3.2 Ergebnisse

2.3.2.1 Relative Visualisierung der Frequenzänderungen

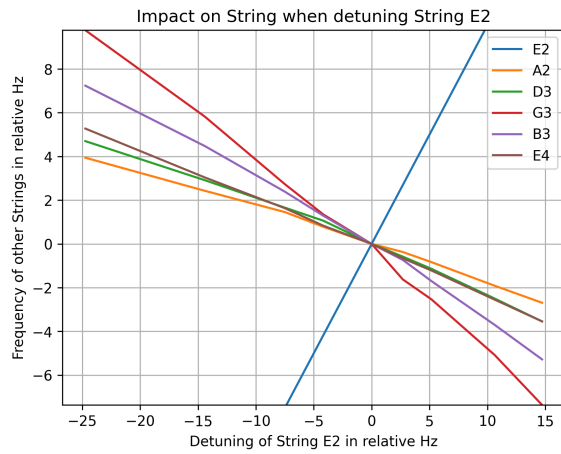


Abbildung 23: Relativer Einfluss der E2 Saite auf die anderen Saiten

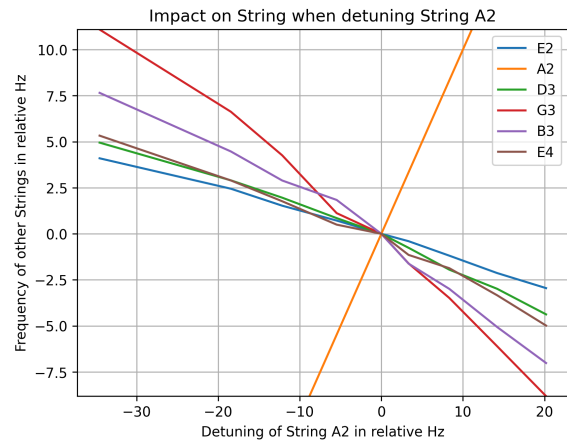


Abbildung 24: Relativer Einfluss der A2 Saite auf die anderen Saiten

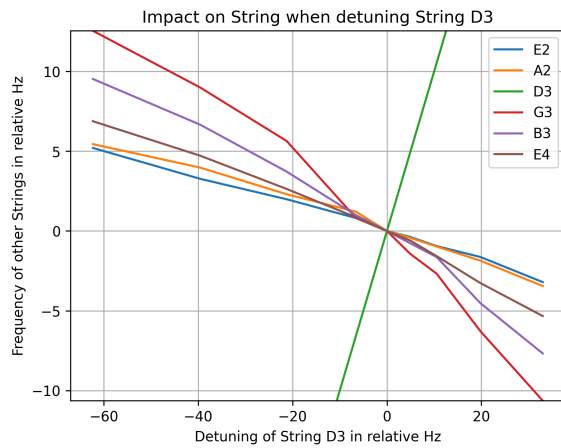


Abbildung 25: Relativer Einfluss der D3 Saite auf die anderen Saiten

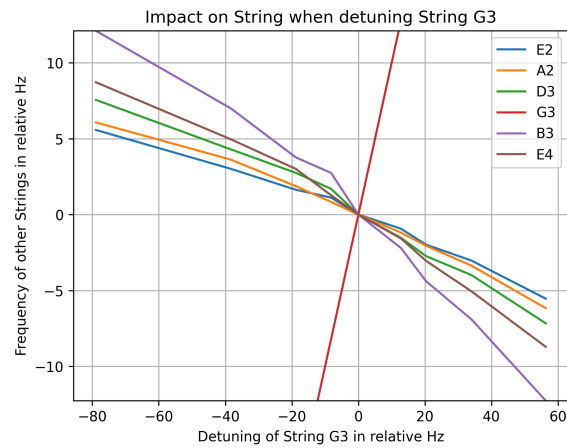


Abbildung 26: Relativer Einfluss der G3 Saite auf die anderen Saiten

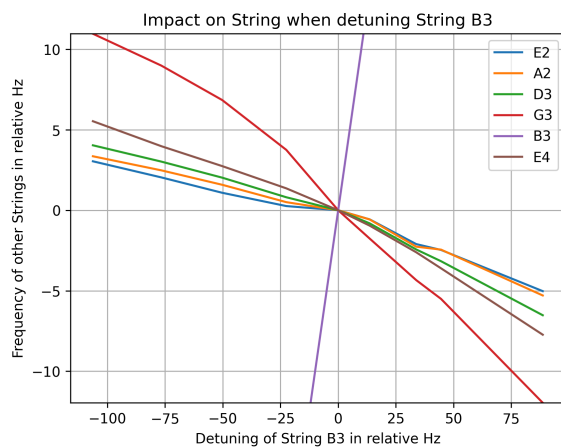


Abbildung 27: Relativer Einfluss der B3 Saite auf die anderen Saiten

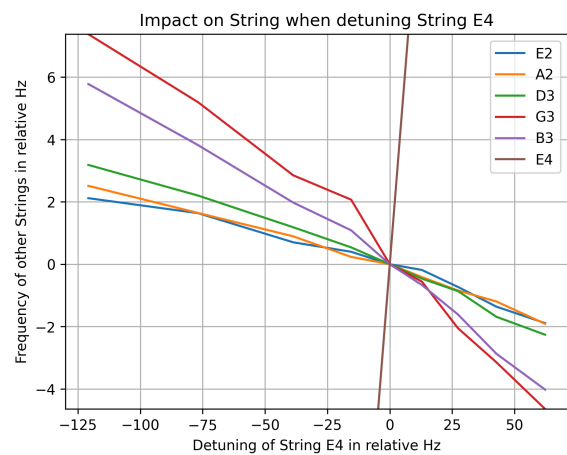


Abbildung 28: Relativer Einfluss der E4 Saite auf die anderen Saiten

2.3.2.2 Korrelationskoeffizienten und Fehler

Pearson Correlation

X values (correlation input)

	E2	A2	D3	G3	B3	E4
E2	1.0	-0.997	-0.997	-0.991	-0.98	-0.982
A2	-0.998	1.0	-0.994	-0.991	-0.983	-0.995
D3	-0.996	-0.993	1.0	-0.99	-0.982	-0.993
G3	-0.998	-0.996	-0.988	1.0	-0.993	-0.992
B3	-0.998	-0.989	-0.99	-0.988	1.0	-0.994
E4	-0.999	-0.993	-0.993	-0.991	-0.987	1.0

Y values (correlation input)

Abbildung 29: Pearson-Korrelationskoeffizienten der Messdaten

Während der Durchführung des Experiments fiel auf, dass beim Zurückbringen einer Saite in ihre Ausgangsposition alle anderen Saiten ebenfalls wieder ihre ursprüngliche Frequenz annahmen.

2.3.3 Diskussion der Ergebnisse

Das System ist elastisch, da Ausgangs- und Endfrequenzen nach jedem Durchgang gleich sind.

Die Linearität des Systems ist nicht perfekt, aber hinreichend gut für kleine Verstimmungen. Sie lässt sich quantitativ mit dem Korrelationskoeffizienten nach Bravais-Pearson [8] zwischen gemessenen und erwarteten Frequenzänderungen jeder Saite bestimmen. In Abbildung 29 sind die Korrelationskoeffizienten für jede Saite dargestellt. Der Betrag aller Werte liegt über 0.98, was auf eine sehr starke positive Korrelation hinweist. Das rechtfertigt die Annahme einer linearen Beziehung für kleine Änderungen.

2.4 Mathematische Lösung

Die Frequenzen der Saiten können als Vektor dargestellt werden:

$$\vec{f}_0 = \begin{pmatrix} f_{E2} \\ f_{A2} \\ f_{D3} \\ f_{G3} \\ f_{B3} \\ f_{E4} \end{pmatrix} \quad (23)$$

$$C = \begin{pmatrix} 1 & c_{12} & c_{13} & c_{14} & c_{15} & c_{16} \\ c_{21} & 1 & c_{23} & c_{24} & c_{25} & c_{26} \\ c_{31} & c_{32} & 1 & c_{34} & c_{35} & c_{36} \\ c_{41} & c_{42} & c_{43} & 1 & c_{45} & c_{46} \\ c_{51} & c_{52} & c_{53} & c_{54} & 1 & c_{56} \\ c_{61} & c_{62} & c_{63} & c_{64} & c_{65} & 1 \end{pmatrix}, \vec{g} = \begin{pmatrix} \hat{f}_{E2} \\ \hat{f}_{A2} \\ \hat{f}_{D3} \\ \hat{f}_{G3} \\ \hat{f}_{B3} \\ \hat{f}_{E4} \end{pmatrix} \quad (24)$$

- $\vec{f}_0$: Ausgangsfrequenzen der Saiten, gemessen z.B. mit einem digitalen Stimmgerät

- C : Verstimmungsmatrix, wobei c_{ij} den Verstimmungsfaktor der Saite i angibt, wenn die Saite j um 1Hz verstimmt wird
- $\vec{g}$: Ziel-Frequenzen nach der Verstimmung

Die Verstimmungsmatrix aus dem Experiment ist in Abbildung 30 dargestellt:

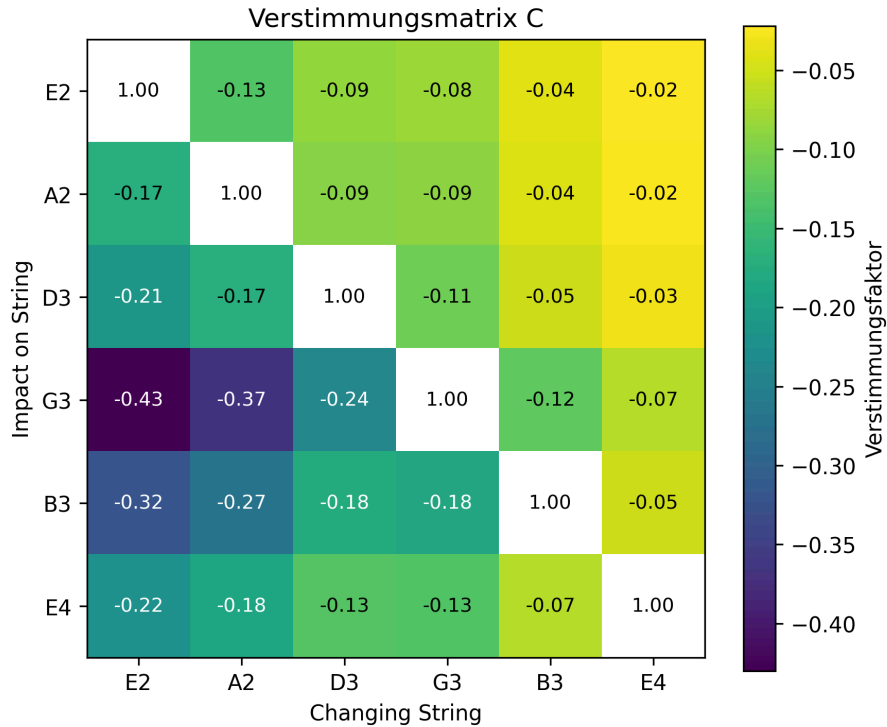


Abbildung 30: Verstimmungsmatrix Beispiel

Der Vektor

$$\vec{\Delta} = \begin{pmatrix} \Delta_{E2} \\ \Delta_{A2} \\ \Delta_{D3} \\ \Delta_{G3} \\ \Delta_{B3} \\ \Delta_{E4} \end{pmatrix} \quad (25)$$

gibt an, um wie viel Hertz jede Saite verstimmt werden muss.

Die effektive Verstimmung wird durch die Multiplikation mit der Verstimmungsmatrix berechnet:

$$C \cdot \vec{\Delta} = \vec{\Delta}_{\text{effective}} \quad (26)$$

Damit die Ziel-Frequenzen $\vec{g}$ erreicht werden, gilt:

$$\vec{g} = \vec{f}_0 + \vec{\Delta}_{\text{effective}} \rightarrow \vec{\Delta}_{\text{effective}} = \vec{g} - \vec{f}_0 \quad (27)$$

Um die Eingangsverstimmung $\vec{\Delta}$ zu bestimmen, muss das Inverse der Matrix C gebildet werden:

$$C \cdot \vec{\Delta} = \vec{\Delta}_{\text{effective}} \rightarrow \vec{\Delta} = C^{-1} \cdot \vec{\Delta}_{\text{effective}} \quad (28)$$

$$\vec{\Delta} = C^{-1} \cdot (\vec{g} - \vec{f}_0) \quad (29)$$

C^{-1} ist die Inverse der Verstimmungsmatrix.

Somit benötigt man für die Berechnung:

1. Ausgangsfrequenzen $\vec{f}_0$
2. Ziel-Frequenzen $\vec{g}$
3. Verstimmungsmatrix C

2.5 Praktische Umwandlung

Für die praktische Anwendung ist die absolute Zielfrequenz jeder Saite entscheidend, da ein Musiker direkt auf einen Frequenzwert stimmen kann, ohne eine mentale Addition durchführen zu müssen.

Da die Saiten sequentiell in der Reihenfolge $E2 \rightarrow A2 \rightarrow D3 \rightarrow G3 \rightarrow B3 \rightarrow E4$ gestimmt werden, beeinflusst jede Verstimmung einer Saite j über die Verstimmungsmatrix C alle nachfolgenden Saiten $i \geq j$. Dieser Kaskadeneffekt muss bei der Berechnung der absoluten Zielfrequenzen berücksichtigt werden. Die Δ_i werden einmalig ausgerechnet. Die sequentielle Formel gibt dann an, auf welche Frequenz Saite N beim Stimmen abzielen ist. Für die erste Saite E2 ergibt sich die Zielfrequenz zu:

$$f_{E2} = f_{0,E2} + \Delta_{E2} \cdot C_{11} \quad (30)$$

Da E2 bereits auf den Sollwert gestimmt wurde, muss dessen Einfluss auf alle nachfolgenden Saiten eingerechnet werden. Die Zielfrequenz von A2 lautet dementsprechend:

$$f_{A2} = f_{0,A2} + \Delta_{A2} \cdot C_{22} + \Delta_{E2} \cdot C_{21} \quad (31)$$

Analog ergibt sich für D3:

$$f_{D3} = f_{0,D3} + \Delta_{D3} \cdot C_{33} + \Delta_{A2} \cdot C_{32} + \Delta_{E2} \cdot C_{31} \quad (32)$$

Dieses Muster lässt sich verallgemeinern. Die absolute Zielfrequenz der N -ten Saite berechnet sich als:

$$f_N = f_{0,N} + \sum_{i=1}^N \Delta_i \cdot C_{N,i} \quad (33)$$

Hierbei wird vorausgesetzt, dass die Saiten stets in aufsteigender Reihenfolge von E2 nach E4 gestimmt werden, sodass die Verstimmungseinflüsse bereits gestimmter Saiten vollständig bekannt sind und in die Berechnung der nachfolgenden Zielfrequenzen einfließen können.

3 Ablauf eines Stimmvorgangs

Im Folgenden wird der Ablauf zum Stimmen einer Floyd-Rose-Gitarre beschrieben sowie die dafür erforderlichen Verfahren erläutert.

3.1 Initiierung – Bestimmung der Verstimmungsmatrix

Vor dem eigentlichen Stimmvorgang muss die gitarrenspezifische Verstimmungsmatrix C einmalig ermittelt werden. Da sie eine physikalische Eigenschaft des jeweiligen Instruments beschreibt, ist diese Kalibrierung nur beim erstmaligen Einsatz der Methode notwendig.

Zur Bestimmung jedes Matrixeintrags werden mindestens zwei Messpunkte benötigt, um die lineare Steigung zu ermitteln. Die Diagonalelemente $C_{ii} = 1$ sind definitionsgemäß bekannt und müssen nicht gemessen werden.

Der erste Messpunkt beschreibt den Ausgangszustand der Gitarre; alle weiteren Messpunkte erfassen das Verhalten der übrigen Saiten bei gezielter Veränderung einer einzelnen Saite. Durch eine höhere Anzahl an Messpunkten lässt sich die Schätzgenauigkeit der Steigung beliebig steigern. Zur Auswertung wird ein lineares Regressionsverfahren eingesetzt.

Dabei kommen folgende Verfahren in Betracht:

Methode der kleinsten Quadrate (OLS) Minimiert die Summe der quadrierten vertikalen Abstände in Y -Richtung zwischen den Datenpunkten und der Regressionsgeraden. Sie setzt voraus, dass ausschließlich die Antwortvariable Y fehlerbehaftet ist, während die Prädiktorvariable X als exakt gilt.

Orthogonale Regression (Deming-Regression) Minimiert die Summe der quadrierten senkrechten Abstände der Datenpunkte zur Regressionsgeraden. Sie wird angewendet, wenn sowohl Y als auch X Messfehler aufweisen. [7]

Da die Frequenzmessungen beider Achsen messtechnisch bedingte Fehler enthalten, wird die **orthogonale Regression** verwendet.

3.2 Stimmvorgang

Zunächst wird die gewünschte Zielstimmung festgelegt. Gitarren werden je nach musikalischem Kontext in verschiedenen Stimmungen gespielt, da bestimmte Akkordgriffe in alternativen Stimmungen vereinfacht oder erst ermöglicht werden.

Anschließend wird der aktuelle Zustand der Gitarre durch eine einmalige Frequenzmessung aller Saiten erfasst, um den Frequenzvektor $\vec{f}_0$ zu bestimmen. Mithilfe von Gleichung 33 wird dann für jede Saite die absolute Zielfrequenz berechnet, auf die sie gestimmt werden muss.

3.3 Überprüfung

Abschließend wird mithilfe eines herkömmlichen Stimmgeräts verifiziert, ob alle Saiten die berechneten Zielfrequenzen erreicht haben und die Gitarre korrekt gestimmt ist.

4 Grundlegende Verfahren

Im Folgenden werden die Verfahren beschrieben, die für die Umsetzung des Stimmvorgangs erforderlich sind.

4.1 Bestimmung der Fundamentalfrequenz

Für die Implementierung eines Stimmgeräts auf mobilen Geräten ist die präzise und effiziente Bestimmung der Grundfrequenz (Fundamental Frequency, F_0) entscheidend. Im Folgenden werden gängige Verfahren vorgestellt und hinsichtlich Genauigkeit, Rechenaufwand und Anwendungsbereich bewertet.

4.1.1 Autokorrelation

Die Grundidee der Autokorrelation beruht darauf, dass ein periodisches Signal mit der Periode τ , wenn es mit einer zeitverschobenen Version seiner selbst multipliziert wird, bei ganzzahligen Vielfachen von τ Maxima aufweist. Das erste von null verschiedene lokale Maximum bestimmt die gesuchte Periode τ ; ihr Kehrwert liefert die Grundfrequenz des Signals.

$$r_t(\tau) = \sum_{j=t+1}^{t+W} x_j \cdot x_{j+\tau} \quad (34)$$

Dabei bezeichnet $r_t(\tau)$ den Autokorrelationswert für die Verzögerung τ zum Zeitindex t und W die Fenstergröße der Integration. [9]

4.1.2 YIN-Algorithmus

Der YIN-Algorithmus stellt eine Weiterentwicklung der klassischen Autokorrelation dar. Durch zusätzliche Fehlerreduktionsschritte wird die Robustheit gegenüber Amplitudenschwankungen verbessert; die sogenannte *Difference Function* filtert dabei unplausible Perioden τ heraus. [9] Aufgrund dieser Eigenschaften findet der Algorithmus breite Anwendung in kommerziellen Stimmgeräten.

4.1.3 Fourier- und Cepstrum-Analyse

Bei der Fourier-Analyse wird das Zeitsignal in den Frequenzbereich transformiert und das resultierende Spektrum nach der Grundfrequenz durchsucht. Studien zeigen jedoch, dass Fourier-basierte Verfahren fehleranfällig sind und hohe Abtastraten erfordern. [10]

Die Cepstrum-Analyse erweitert diesen Ansatz, indem das logarithmierte Spektrum erneut transformiert wird, um periodische Muster zu detektieren. Ein Vorteil ist die Robustheit gegenüber harmonischen Obertönen sowie die gute Integration in digitale Signalverarbeitungssysteme. Nachteilig wirkt sich die eingeschränkte Genauigkeit bei niedrigen Frequenzen und verrauschten Signalen aus, da das Verfahren Periodizität und harmonische Obertöne voraussetzt – Annahmen, die in realen Umgebungen nicht immer erfüllt sind. [11]

4.1.4 Moderne Deep-Learning-Ansätze

Neuronale Netze wie CREPE oder DeepPitch nutzen Convolutional Neural Networks oder Recurrent Neural Networks, um die Grundfrequenz direkt aus Roh-Audio oder Spektrogrammen zu schätzen. Vorteile sind die hohe Robustheit gegenüber Polyphonie, Hintergrundgeräuschen und verschiedenen Instrumenten. Nachteilig sind der erhebliche Rechenaufwand, der Bedarf an großen Trainingsdatensätzen sowie die eingeschränkte Eignung für ressourcenbeschränkte Mobilgeräte. [12]

4.1.5 Auswahl des Verfahrens

Für die Implementierung wird der YIN-Algorithmus gewählt, da er eine ausgewogene Balance zwischen Genauigkeit und Rechenaufwand bietet. Er ist speziell für die Schätzung monophoner Grundfrequenzen entwickelt worden, zeigt robuste Leistung unter variierenden Signalbedingungen und ist in zahlreichen Programmiersprachen bereits als Bibliothek verfügbar.

4.2 Signal-Filter

Um die Zuverlässigkeit der Grundfrequenzschätzung zu erhöhen, wird Ein- und Ausgangssignal gefiltert und konditioniert.

4.2.1 Bandpassfilterung durch Parameteranpassung

Da der YIN-Algorithmus verwendet wird, lässt sich eine implizite Bandpassfilterung durch gezielte Anpassung seiner Parameter erreichen. Durch Reduktion der Abtastrate werden Frequenzen oberhalb der halben Nyquist-Frequenz nicht mehr erfasst, was dem Nyquist-Shannon-Abtasttheorem entspricht. [13]

Die untere Grenzfrequenz wird durch die maximale Fensterlänge W begrenzt. Gemäß $f = 1/T$ darf die Fensterlänge den Wert $T_{\max} = 1/f_{\min}$ ¹ nicht überschreiten. Da der Frequenzbereich von Gitarrensaiten näherungsweise 50 Hz bis 350 Hz umfasst, lassen sich die Parameter entsprechend dimensionieren.

¹Mit „min“ ist das Minimum gemeint.

4.2.2 Gleitender Mittelwert für Streaming-Messdaten

Mikrofonaufnahmen enthalten unvermeidlich Umgebungsgeräusche, die zu Schwankungen in der Frequenzschätzung führen. Um eine stabile Visualisierung zu gewährleisten, wird ein gleitender Mittelwert über die letzten N Messwerte gebildet und angezeigt. Dieses Verfahren reduziert kurzzeitige Ausreißer, ohne dabei wesentliche Latenzen einzuführen.

4.2.3 Amplitudenschwelle

Um eine kontinuierliche Grundfrequenzschätzung bei Stille oder Umgebungslärm zu vermeiden, wird die Schätzung nur aktiviert, wenn das Signal einen Mindestpegel überschreitet. Der Schalldruckpegel wird dabei gemäß

$$L = 20 \cdot \log_{10} \left(\frac{\text{RMS}}{32768} \right) \quad (35)$$

in *decibels relative to full scale* (dBFS) [14] berechnet, wobei 32768 dem maximalen Amplitudenwert eines 16-Bit-PCM-Signals entspricht.

5 Software Entwicklung/Implementierung

Die Methodik der Softwareentwicklung wurde durch das Buch „Mobile App Engineering“ [15] inspiriert.

Das Buch beschäftigt sich mit der Entwicklung von *Enterprise Apps*. Die in diesem Rahmen entwickelte App ist zwar keine *Enterprise App*, aber die Prinzipien der Softwareentwicklung, die in diesem Buch beschrieben werden, sind dennoch anwendbar. Es werden insbesondere die Prinzipien der Anforderungsanalyse und der nutzerzentrierten Gestaltung übernommen.

Wie im Buch beschrieben, werden Mobile Applikationen in iterativen Prozessen entwickelt. Daher werden manche Designentscheidungen mit Usertests begründet, die mit einer älteren Version der App durchgeführt wurden.

5.1 Requirements Engineering

Dieses Kapitel wurde mit Hilfe des Buchs „Mobile App Engineering“ [15] entwickelt. Die Ergebnisse folgen aus der Befolgung des Kapitel 4 „Requirements Engineering“.

5.1.1 Projektvision

Ziel des Projekts ist die Entwicklung einer mobilen Applikation, die Gitarristen beim effizienten Stimmen von Gitarren mit Floyd-Rose-Brücke unterstützt und den damit verbundenen zeitlichen sowie technischen Aufwand minimiert.

5.1.2 Ziel- und Benutzergruppe

Die Zielgruppe umfasst alle Personen, die regelmäßig oder gelegentlich eine Gitarre mit Floyd-Rose-Brücke stimmen müssen. Sie lässt sich in drei Segmente unterteilen:

Gitarren-Einsteiger erwerben eine Floyd-Rose-Gitarre häufig ohne vollständiges Bewusstsein über den erhöhten Stimmaufwand dieses Brückensystems. Vorkenntnisse über Gitarrentechnik sind in dieser Gruppe gering; Alter und technisches Vorwissen variieren stark.

Professionelle Gitarristen verfügen über mehrere Instrumente in unterschiedlichen Stimmungen, um ein breites klangliches Spektrum abdecken zu können. Sie besitzen fundierte Gitarrenkenntnisse und kommunizieren in der Regel auf Englisch.

Gitarrentechniker in Musikgeschäften stellen die zeitkritischste Benutzergruppe dar. Sie stimmen und warten potenziell täglich mehrere Floyd-Rose-Gitarren für Kunden, verfügen über tiefgehendes technisches Fachwissen und sind mit englischsprachiger Fachterminologie vertraut. Für diese Gruppe bietet die Applikation den größten Effizienzgewinn.

5.1.3 Systemkontext

Der Systemkontext beschreibt alle materiellen und immateriellen Einflussfaktoren, die im Zusammenhang mit dem Projekt stehen. [15] Im vorliegenden Fall umfasst er folgende Aspekte:

Benutzer: Personen, die eine Floyd-Rose-Gitarre stimmen möchten, unabhängig von ihrem technischen Kenntnisstand.

Akustische Umgebung: Die Umgebungsgeräusche am Ort des Stimmvorgangs beeinflussen die Qualität der Frequenzmessung und stellen eine zentrale Einflussgröße für die Zuverlässigkeit der Applikation dar.

Hardware des Mobilgeräts: Die Mikrofonqualität des verwendeten Endgeräts wirkt sich direkt auf die Genauigkeit der Grundfrequenzschätzung aus.

Spielweise der Gitarre: Es ist zu unterscheiden, ob die Gitarre über einen Verstärker mit ausreichendem Schallpegel betrieben oder unverstärkt und damit leiser angespielt wird. Zudem kann ein ausgeprägter Obertonanteil die Frequenzanalyse erschweren.

Da die Applikation vollständig lokal auf dem Endgerät ausgeführt wird, ist kein IT-Backend und keine Internetverbindung erforderlich.

5.1.4 Anwendungskontext

Der Anwendungskontext wurde maßgeblich durch die in Abschnitt 6.3 dokumentierten Nutzertests ermittelt. Dabei wurden folgende praxisrelevante Erkenntnisse gewonnen:

Akustische Störeinflüsse: Ein Teil der Testpersonen stimmte die Gitarre in lauten Umgebungen oder spielte Instrumente mit ausgeprägtem Obertonanteil. Beides führte zu einer fehlerhaften Erkennung der Grundfrequenz und stellt damit eine zentrale Herausforderung für die Signalverarbeitung dar.

Benutzerführung: Die Tests zeigten, dass rein textbasierte Erklärungen für einen Teil der Nutzer nicht ausreichen. Visuelle Unterstützung durch Illustrationen oder animierte Hinweise ist notwendig, um Bedienabläufe verständlich zu vermitteln.

Darstellung von Messwerten: Numerische Frequenzangaben erzeugten bei Testpersonen häufig Verwirrung. Eine abstrahierte, intuitiv verständliche Visualisierung der Stimmgenauigkeit ist daher einer rein zahlenwertbasierten Darstellung vorzuziehen.

Abgrenzung des Anwendungsbereichs: Einzelne Testpersonen äußerten den Wunsch nach einer Integration in professionelle Musikstudio-Umgebungen sowie nach einer Verwendbarkeit als Plugin für Digital Audio Workstations (DAW). Dieser Anwendungsfall liegt jedoch außerhalb des definierten Projektumfangs und wird nicht berücksichtigt.

5.1.5 Personas

Um die Anforderungen der Zielgruppe greifbar zu machen, wurden auf Basis der Zielgruppensegmente vier repräsentative Personas entwickelt.

5.1.5.1 Emil – Gitarren-Einsteiger

Emil ist 26 Jahre alt, arbeitet als Kellner und macht Musik als Hobby. Er hat Translationswissenschaften studiert und keinen technischen Hintergrund. Durch ein Familiengeschenk kam er in den Besitz einer

Floyd-Rose-Gitarre, ohne sich zuvor mit den Besonderheiten dieses Brückensystems auseinanderzusetzen zu haben.

Emil nutzt alltäglich Spotify, WhatsApp, Instagram und YouTube. Er liest Texte selten vollständig und bevorzugt visuelle oder interaktive Inhalte. Er ist ungeduldig und erwartet, dass Anwendungen ihn intuitiv durch Prozesse führen.

Der erhöhte Zeitaufwand beim Stimmen frustriert ihn zunehmend und lässt ihn über den Kauf einer einfacher zu stimmenden Gitarre nachdenken.

Relevanz: Emil repräsentiert Nutzer mit geringem Vorwissen, die eine niedrige Einstiegshürde und eine geführte Benutzeroberfläche benötigen.

5.1.5.2 Matilda – Professionelle Gitarristin

Matilda ist 38 Jahre alt und verdient ihren Lebensunterhalt als Gitarristin ihrer Metalband „Fire Hawks„. Sie spielt seit 20 Jahren E-Gitarre und besitzt eine Sammlung mehrerer Instrumente, darunter ihre bevorzugte Music Man Silhouette mit Floyd-Rose-Tremolo.

Da sie ihre Saiten aufgrund intensiver Bespielung regelmäßig wechselt, ist das Neustimmen für sie Routine. Matilda wechselt häufig zwischen Standard- und Drop-D-Stimmung und schätzt dabei Effizienz. Sie kommuniziert per E-Mail, Telefon und WhatsApp; als Musikerin nutzt sie bewusst Streaming-Plattformen, die Künstler stärker vergüten als marktführende Dienste.

Relevanz: Matilda repräsentiert erfahrene Nutzerinnen, die schnelle Workflows und die Unterstützung mehrerer Stimmungen priorisieren.

5.1.5.3 Jonas – Gitarrentechniker

Jonas ist 45 Jahre alt und arbeitet seit 18 Jahren als Gitarrentechniker in einem Musikfachgeschäft. Er wartet, repariert und stimmt täglich Instrumente verschiedener Kunden – darunter regelmäßige Gitarren mit Floyd-Rose-Brücke.

Er verfügt über tiefgehendes technisches Fachwissen zu Gitarren aller Bauarten und ist mit englischsprachiger Fachterminologie vertraut. Zeiteffizienz hat für ihn höchste Priorität, da er unter Umständen mehrere Floyd-Rose-Gitarren pro Tag stimmen muss. Er ist gegenüber neuen Werkzeugen aufgeschlossen, sofern sie seinen Workflow beschleunigen und zuverlässig funktionieren.

Relevanz: Jonas repräsentiert professionelle Anwender im gewerblichen Umfeld, für die die Applikation den größten messbaren Effizienzgewinn liefert.

5.1.5.4 Hanna – Home-Producerin

Hanna ist 30 Jahre alt und betreibt ein eigenes Heimstudio, in dem sie regelmäßig Bands aufnimmt und mischt – darunter die Band „Fire Hawks„. Sie arbeitet mit der *DAW Bitwig* und ist mit *CLAP*- und *VST-Plugin-Formaten* sowie den Grundlagen der digitalen Signalverarbeitung vertraut. Darüber hinaus programmiert sie eigenständig und verfügt über hochpräzise Frequenzmessmöglichkeiten in ihrer Studioumgebung.

Im Aufnahmekontext begegnen ihr regelmäßig Gitarristinnen und Gitarristen mit Floyd-Rose-Instrumenten. Da für eine qualitativ hochwertige Aufnahme eine präzise Stimmung unerlässlich ist, hat der Stimmvorgang in der Vergangenheit wiederholte Aufnahmesessions verzögert. Hanna ist mit einschlägiger Fachterminologie aus Tontechnik und Gitarrenbau vertraut und erwartet von Werkzeugen eine hohe Messgenauigkeit.

Relevanz: Hanna repräsentiert technisch versierte Nutzerinnen im professionellen Aufnahmekontext, für die Präzision und Geschwindigkeit des Stimmvorgangs direkte Auswirkungen auf den Produkti-

onsablauf haben. Zudem verfügen Nutzerinnen dieses Segments außerhalb der Applikation über hochpräzise Messinstrumente, was eine entsprechend hohe Messgenauigkeit voraussetzt.

5.1.6 Szenarien

5.1.6.1 Szenario 1 – Emils erster Versuch

Normaler Ablauf: Emil kommt nach einem langen Arbeitstag nach Hause und möchte zur Entspannung Gitarre spielen. Über eine YouTube-Empfehlung wird er auf die Applikation aufmerksam und hofft, damit den Kauf einer neuen Gitarre vermeiden zu können. Er lädt die App herunter, setzt sich mit der Gitarre auf den Boden seines Wohnzimmers und legt das Smartphone vor sich.

Nach dem Start der Applikation kann er eine Zielstimmung sowie ein Gitarrenprofil auswählen. Da noch kein Profil existiert, legt er über einen prominenten Button eine neue Gitarre an und benennt sie „Onkel Ullies Gitarre,“. Anschließend gelangt er in den Kalibrierungsmodus, in dem er aufgefordert wird, jede Saite einzeln anzupspielen. Daraufhin wird er gebeten, die E2-Saite gezielt zu verstimmen und alle Saiten erneut zu messen. Eine Fortschrittsanzeige informiert ihn über die verbleibenden Messschritte. Nach Abschluss der Kalibrierung ist das Gitarrenprofil angelegt.

Im anschließenden Stimmvorgang misst Emil zunächst den aktuellen Zustand der Gitarre. Die App zeigt ihm daraufhin für jede Saite eine visuelle Zielanzeige, anhand derer er die Saite in den grünen Bereich stimmt. Nach Abschluss aller Saiten wird ihm im integrierten Stimmgerät bestätigt, dass die Gitarre korrekt gestimmt ist.

Mögliche Fehler und Lösungsansätze:

1. **Falsche Stimmung gewählt:** Emil wählt versehentlich eine unbekannte Stimmung aus und ist anschließend verwirrt, da die Gitarre nicht dem erwarteten Klang entspricht. Da Emil textbasierte Erklärungen meidet, sollte ein kurzes Erklärvideo zur Verfügung stehen; ergänzend kann ein knapper Infotext eingeblendet werden.
2. **Unbekannte Saitenbezeichnungen:** Emil ist mit den Bezeichnungen E2, A2, D3, G3, B3 und E4 nicht vertraut. Auch hier bietet ein kurzes Demonstrationsvideo den geeigneten Einstieg.
3. **Schwierigkeiten bei der Frequenzeingabe:** Emil weiß nicht, wie er die Frequenz einer Saite in die App einträgt. Ein Anleitungsvideo, das den Messvorgang vorführt, ermöglicht ihm ein einfaches Nachahmen.
4. **Falsche Saite verstimmt:** Emil verstimmt versehentlich die falsche Saite, ohne es zu bemerken. Die Applikation sollte anhand der gemessenen Frequenzen automatisch erkennen, ob die korrekte Saite verstimmt wurde, und den Nutzer gegebenenfalls zur Wiederholung auffordern.
5. **Frequenz nicht erkennbar:** Die Gitarre wird zu leise angespielt, sodass das Mikrofon – insbesondere bei tiefen Saiten – kein zuverlässiges Signal erfasst. In diesem Fall werden häufig harmonische Obertöne gemessen; die App sollte durch Rückrechnung die Grundfrequenz ableiten und eine Plausibilitätsprüfung anhand des erwarteten Frequenzbereichs durchführen.
6. **Defektes Mikrofon:** Das Mikrofon des Endgeräts ist nicht funktionsfähig. Als Fallback sollte die manuelle Eingabe von Frequenzwerten über ein Textfeld ermöglicht werden.

5.1.6.2 Szenario 2 – Matilda auf einer Jam-Session

Normaler Ablauf: Matilda spielt auf einer Jam-Session und möchte für den nächsten Auftritt die Stimmung ihrer Gitarre wechseln. Sie öffnet die App, wählt die gewünschte Zielstimmung Drop-D sowie ihr gespeichertes Gitarrenprofil aus und startet den Stimmvorgang. Da die Umgebung laut

ist, benötigt sie mehrere Versuche, um stabile Frequenzmessungen zu erhalten. Nach erfolgreichem Abschluss aller Saiten ist die Gitarre korrekt gestimmt.

Mögliche Fehler und Lösungsansätze (ergänzend zu Szenario 1):

1. **Störgeräusche unterbrechen die Messung:** Hintergrundgeräusche führen dazu, dass kontinuierlich neue Frequenzwerte erkannt werden, ohne dass eine Saite angespielt wurde. Über einen einstellbaren Lautstärkeschwellenwert – etwa einen Schieberegler in den Einstellungen – kann die Empfindlichkeit des Mikrofons so angepasst werden, dass Messungen nur ausgelöst werden, wenn die Lautstärke der gespielten Saite den Umgebungspegel deutlich übersteigt.

5.1.7 Anforderungen

Auf Basis der Projektvision, des System- und Anwendungskontexts sowie der erstellten Personas und Anwendungsszenarien wurden funktionale und nicht-funktionale Anforderungen an die Applikation abgeleitet. Wie in [15] empfohlen, werden diese nach *MUSS*- und *SOLL*-Kriterien sowie nach der Wertigkeit gemäß Kano-Modell priorisiert. Anforderungen mit hohem Wert und hohem Ausfallrisiko erhalten dabei die höchste Priorität, gefolgt von Anforderungen mit hohem Nutzen bei geringem Risiko. Anforderungen mit geringem Nutzen und geringem Risiko werden zuletzt eingestuft. Dies folgt aus [15] und der sogenannten *Wert-Risiko-Matrix*[16].

5.1.7.1 Funktionale Benutzeranforderungen

FBA-01	Ein Nutzer muss eine Floyd-Rose-Gitarre effizient stimmen können.	<i>MUSS</i>
FBA-02	Ein Nutzer muss die App auf seine Gitarre kalibrieren können.	<i>MUSS</i>
FBA-03	Ein Nutzer muss die Kalibrierung mehrerer Gitarren langfristig speichern können.	<i>MUSS</i>
FBA-04	Ein Nutzer muss den aktuellen Stimmzustand seiner Gitarre messen können.	<i>MUSS</i>
FBA-05	Ein Nutzer muss die Empfindlichkeit der Frequenzmessung einstellen können, um die App in geräuschbelasteten Umgebungen zuverlässig nutzen zu können.	<i>MUSS</i>
FBA-06	Ein Nutzer muss Hilfe zur Bedienung der App erhalten können.	<i>MUSS</i>
FBA-07	Ein Nutzer muss die Korrektheit einer Messung überprüfen können, sofern dies nicht bereits automatisch durch die App erfolgt.	<i>MUSS</i>
FBA-08	Ein Nutzer muss fehlerhafte Messungen korrigieren können.	<i>MUSS</i>
FBA-09	Ein Nutzer sollte unterschiedliche Stimmungen für den Stimmprozess auswählen können.	<i>SOLL</i>
FBA-10	Ein Nutzer sollte die Stimmung seiner Gitarre mithilfe eines herkömmlichen Stimmgeräts auf Richtigkeit prüfen können.	<i>SOLL</i>

FBA-11	Ein Nutzer sollte die App auf seine Gitarre rekalisieren können.	<i>SOLL</i>
FBA-12	Ein Nutzer sollte Kalibrierungen einen benutzerdefinierten Namen vergeben können.	<i>SOLL</i>
FBA-13	Ein Nutzer sollte eigene Stimmungen erstellen, bearbeiten und löschen können.	<i>SOLL</i>
FBA-14	Ein Nutzer sollte Messungen auch manuell mit externen Hilfsmitteln (z. B. einem separaten Frequenzmessgerät) vornehmen und die Werte eintragen können.	<i>SOLL</i>
FBA-15	Ein Nutzer sollte nicht allgemein bekannte Begriffe – insbesondere Stimmungsbezeichnungen und Saitennamen – nachvollziehen können.	<i>SOLL</i>
FBA-16	Ein Nutzer sollte die Präzision des Stimmungsprozesses erhöhen können	<i>SOLL</i>

5.1.7.2 Funktionale Systemanforderungen

FSA-01	Das System muss die Grundfrequenz einer angespielten Saite mittels YIN-Algorithmus in Echtzeit schätzen.	<i>MUSS</i>
FSA-02	Das System muss Frequenzmessungen nur durchführen, wenn der Schalldruckpegel des Eingangssignals einen konfigurierbaren dBFS-Schwellenwert überschreitet.	<i>MUSS</i>
FSA-03	Das System muss nach einer vollständigen Zustandsmessung aller Saiten ($\vec{f}_0$) für jede Saite N die absolute Zielfrequenz gemäß $f_N = f_{0,N} + \sum_{i=1}^N \Delta_i \cdot C_{N,i} \quad (36)$ berechnen.	<i>MUSS</i>
FSA-04	Das System muss sicherstellen, dass Saiten stets in der Reihenfolge E2 $\rightarrow$ A2 $\rightarrow$ D3 $\rightarrow$ G3 $\rightarrow$ B3 $\rightarrow$ E4 gestimmt werden, sodass die Verstimmungseinflüsse bereits gestimmter Saiten in die Zielfrequenz nachfolgender Saiten einfließen.	<i>MUSS</i>
FSA-05	Das System muss einen gleitenden Mittelwert über die letzten N Messwerte berechnen und zur Anzeige verwenden, um kurzfristige Ausreißer zu dämpfen.	<i>MUSS</i>

FSA-06	Das System muss den Frequenzbereich der Schätzung auf näherungsweise 50 Hz bis 350 Hz begrenzen.	<i>MUSS</i>
FSA-07	Das System muss bei der Kalibrierung für jede Saite j mindestens zwei Messpunkte erfassen und daraus den Eintrag $C_{i,j}$ der Verstimmungsmatrix mittels orthogonaler Regression (Deming-Regression) schätzen.	<i>MUSS</i>
FSA-08	Das System muss die Diagonalelemente $C_{ii} = 1$ der Verstimmungsmatrix ohne Messung als bekannt voraussetzen.	<i>MUSS</i>
FSA-09	Das System muss bei der Kalibrierung automatisch prüfen, ob die vom Nutzer verstimmte Saite der geforderten Saite entspricht, und bei Abweichung die Messung verwerfen und den Schritt wiederholen.	<i>MUSS</i>
FSA-10	Das System muss die gitarrenspezifische Verstimmungsmatrix C persistent auf dem Gerät speichern und beim nächsten Start wiederherstellen.	<i>MUSS</i>
FSA-11	Das System muss die Funktionalität eines Standard-Stimmgeräts anbieten, um die Korrektheit der Stimmung zu verifizieren.	<i>MUSS</i>
FSA-12	Das System sollte bei einer gemessenen Frequenz, die einem harmonischen Oberton entspricht, durch Halbierung auf die Grundfrequenz rückschließen und den Wert anhand des erwarteten Frequenzbereichs der jeweiligen Saite plausibilisieren.	<i>SOLL</i>
FSA-13	Das System sollte für jedes Gitarrenprofil einen benutzerdefinierten Namen persistent speichern und beim Laden anzeigen.	<i>SOLL</i>
FSA-14	Das System sollte vordefinierte Stimmungen (mindestens Standard-E und Drop-D) als unveränderliche Referenzwerte bereitstellen.	<i>SOLL</i>
FSA-15	Das System sollte benutzerdefinierte Stimmungen persistent speichern, bearbeiten und löschen können.	<i>SOLL</i>
FSA-16	Das System sollte das Erfassen von mehr als zwei Messpunkten pro Saite ermöglichen, um die Schätzgenauigkeit der Verstimmungsmatrix durch zusätzliche Stützstellen für die Deming-Regression zu erhöhen.	<i>SOLL</i>

5.1.7.3 Nichtfunktionale Anforderungen

5.1.7.3.1 Messgenauigkeit

NFA-MG-01	Das System muss die Grundfrequenz einer Gitarrensaite mit einer Abweichung von maximal ± 0.2 Hz vom wahren Wert schätzen, gemessen unter kontrollierten akustischen Bedingungen.	<i>MUSS</i>
NFA-MG-02	Das System muss harmonische Obertöne von der Grundfrequenz unterscheiden und darf diese nicht als Grundfrequenz ausgeben, sofern der Signalpegel des Grundtons den Schwellenwert überschreitet.	<i>MUSS</i>
NFA-MG-03	Das System sollte auch bei unverstärktem Spiel eine Messgenauigkeit von ± 0.1 Hz einhalten. ²	<i>SOLL</i>

5.1.7.3.2 Latenz

NFA-LA-01	Das System muss die visuelle Zielanzeige innerhalb von 100 ms nach Eingang eines stabilen Messwerts aktualisieren, sodass der Nutzer beim Stimmen unmittelbares Feedback erhält.	<i>MUSS</i>
NFA-LA-02	Das System muss die Berechnung der Zielfrequenzen aller sechs Saiten nach Abschluss der Zustandsmessung in unter 500 ms abschließen.	<i>MUSS</i>

5.1.7.3.3 Robustheit

NFA-RO-01	Das System muss bei Umgebungsgeräuschen bis zu einem Pegel von 70 dB(A) stabile Frequenzmessungen liefern, sofern der Schallpegel der gespielten Saite den Umgebungspegel um mindestens 10 dB übersteigt.	<i>MUSS</i>
NFA-RO-02	Das System darf bei Stille oder reinem Umgebungslärm keine Frequenzschätzung ausgeben und muss in diesem Zustand die Zielanzeige nicht aktualisieren.	<i>MUSS</i>
NFA-RO-03	Das System muss bei einem nicht funktionsfähigen Mikrofon in einen Fallback-Modus wechseln und den Nutzer darüber informieren.	<i>MUSS</i>

²Da Töne logarithmisch wahrgenommen werden, steigt die geforderte absolute Genauigkeit mit sinkender Frequenz.

5.1.7.3.4 Bedienbarkeit

NFA-BE-01	Das System muss so gestaltet sein, dass ein Nutzer ohne Vorkenntnisse im Umgang mit Floyd-Rose-Gitarren den vollständigen Erststart – Kalibrierung und erster Stimmvorgang – ohne externe Hilfe abschließen kann.	<i>MUSS</i>
NFA-BE-02	Das System muss alle zentralen Aktionen (Profil auswählen, Stimmvorgang starten, Messung auslösen) mit maximal drei Interaktionen erreichbar machen.	<i>MUSS</i>
NFA-BE-03	Das System sollte Bedienabläufe durch visuelle Mittel (Illustrationen, Animationen) vermitteln und auf rein textbasierte Anleitungen verzichten, wo visuelle Alternativen verfügbar sind.	<i>SOLL</i>
NFA-BE-04	Das System sollte kurze Erklärvideos zu Saitenbezeichnungen, Stimmungswahl und Messvorgang bereitstellen.	<i>SOLL</i>

5.1.7.3.5 Kompatibilität

NFA-KO-01	Das System muss auf iOS (ab Version 16) und Android (ab Version 10) lauffähig sein.	<i>MUSS</i>
NFA-KO-02	Das System muss auf Geräten mit einem Arbeitsspeicher von mindestens 2 GB ohne merkbare Leistungseinbußen betrieben werden können.	<i>MUSS</i>
NFA-KO-03	Das System sollte auf gängigen Gerätegrößen (4,7 Zoll bis 6,7 Zoll Bildschirmdiagonale) ohne Layoutbrüche dargestellt werden.	<i>SOLL</i>

5.1.7.3.6 Datenschutz und Betrieb

NFA-DP-01	Das System darf keine Nutzerdaten, Audiodaten oder Gitarrenprofile an externe Server übermitteln; alle Daten verbleiben ausschließlich lokal auf dem Endgerät.	<i>MUSS</i>
NFA-DP-02	Das System muss ohne aktive Internetverbindung vollständig funktionsfähig sein.	<i>MUSS</i>
NFA-DP-03	Das System sollte gespeicherte Gitarrenprofile und Stimmungen bei einer Neuinstallation durch ein Export-/Importformat wiederherstellbar machen.	<i>SOLL</i>

5.1.7.4 Ausgeschlossene Anforderungen

Die folgenden Anwendungsfälle liegen explizit außerhalb des definierten Projektumfangs und werden nicht berücksichtigt:

- Integration in professionelle Musikstudio-Umgebungen
- Verwendbarkeit als Plugin für Digital Audio Workstations (DAW)
- Netzwerkbasierte Funktionen wie Cloud-Synchronisation oder Mehrgeräte-Unterstützung

5.1.7.5 Hinweis

Da ein vollständiges Anforderungsdokument den Rahmen dieser Bachelorarbeit sprengen würde, wurde auf Formalien wie Projektteam- und Rollenbeschreibung, *Storyboards*, *UML-Anwendungsfalldiagramme*, *Anwendungsfallschablonen*, ausformulierte *User Stories* sowie die Einhaltung von Barrierefreiheitsstandards und weiteren Normen verzichtet.

5.1.8 Entwicklungsparadigma

Die Applikation richtet sich gemäß NFA-KO 1 an Nutzer beider marktführenden mobilen Betriebssysteme – iOS und Android. Eine native Entwicklung für jede Plattform separat würde bedeuten, dieselbe Fachlogik – insbesondere den YIN-Algorithmus (FSA 1), die Berechnung der Verstimmungsmatrix (FSA 7) sowie die Zielfrequenzformel (FSA 3) – doppelt zu implementieren und zu warten.

Da die Applikation gemäß NFA-DP 1 und NFA-DP 2 vollständig lokal und ohne Netzwerkzugriff betrieben wird, entfallen die Hauptargumente gegen Cross-Platform-Ansätze: Es gibt keine plattform-spezifischen Push-Benachrichtigungen, keine hardwarenahen Hintergrunddienste und keine nativen Zahlungsschnittstellen. Der einzige plattformnahe Zugriff – das Gerätmikrofon – gehört zu den am weitesten verbreiteten hardwarenahen Features mobiler Geräte und wird von allen gängigen Cross-Compiling-Frameworks über eine stabile, plattformübergreifende API abgedeckt. Der Effizienzgewinn einer gemeinsamen Codebasis überwiegt daher den Mehraufwand einer rein nativen Implementierung deutlich.

5.2 Konzeption und Design

5.2.1 Informationsarchitektur

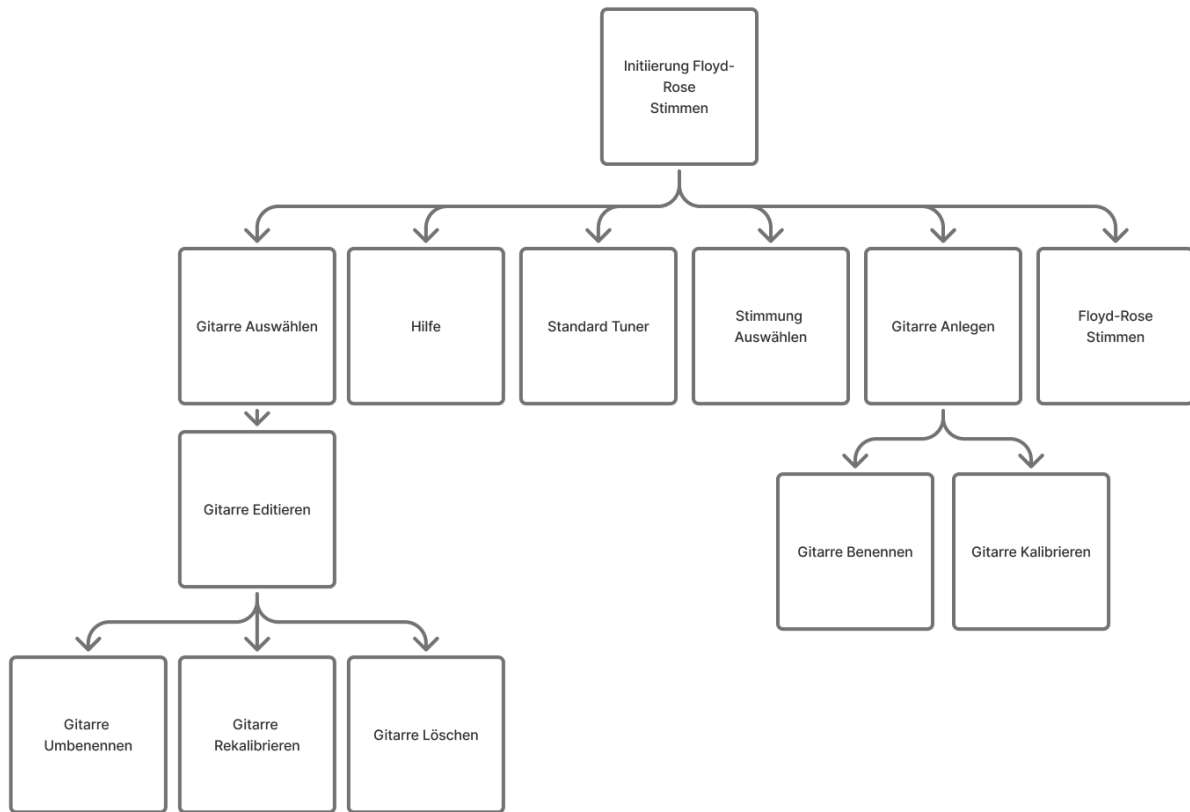


Abbildung 31: Informationsarchitektur der Floyd-Rose-Tuner-App

Abbildung 31 zeigt die Informationsarchitektur der App. Den zentralen Einstiegspunkt bildet der Startbildschirm, von dem aus alle wesentlichen Funktionen erreichbar sind:

1. **Hilfe** – kontextsensitive Unterstützung zur Bedienung (FBA 6)
2. **Gitarre auswählen** – Laden eines gespeicherten Gitarrenprofils für den Stimmvorgang (FBA 3)
3. **Standard-Stimmgerät** – konventionelles chromatisches Stimmgerät zur schnellen Überprüfung der Stimmung ohne Floyd-Rose-Logik (FBA 10)
4. **Stimmung auswählen** – Zielstimmung für den Stimmprozess festlegen (FBA 9)
5. **Gitarre anlegen** – neues Gitarrenprofil erstellen; relevant insbesondere für Persona Jonas (Abschnitt 5.1.5.3), der täglich neue Instrumente stimmt (FBA 2)
6. **Floyd-Rose-Stimmen** – Kernfunktion der App (FBA 1)

Wird eine bestehende Gitarre ausgewählt, stehen zusätzlich die Aktionen *Umbenennen*, *Rekalibrieren* (FBA 11) und *Löschen* zur Verfügung. Wird eine neue Gitarre angelegt, durchläuft der Nutzer unmittelbar die Benennung und den Kalibrierungsvorgang (FBA 12, FBA 2).

5.2.2 Interaktionsdesign

Für die Erstellung des Interaktionsdesigns wurde die grafische Sprache *Visuelles Vokabular* nach [17] verwendet. Die resultierenden Diagramme sind in Abbildung 32, Abbildung 33 und Abbildung 34 dargestellt.

5.2.2.1 Navigation

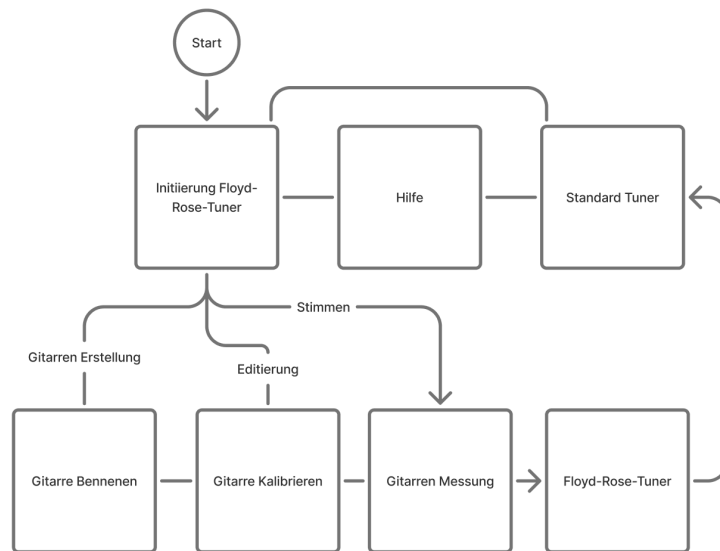


Abbildung 32: Interaktionsdesign – Navigation

Den Einstiegspunkt bildet die Floyd-Rose-Initiierungsseite, von der aus der Nutzer die Hilfeseite, das Standard-Stimmgerät, die Gitarrenansicht sowie den Stimmprozess erreichen kann.

Vom Standard-Stimmgerät aus ist es möglich, direkt in die Kalibrierung der aktuell ausgewählten Gitarre zu wechseln, falls das Ergebnis nicht den Erwartungen entspricht (FBA 11). Auf der Gitarrenansichtsseite kann die Gitarre umbenannt (FBA 12) und rekaliert werden.

5.2.2.2 Kalibrierung

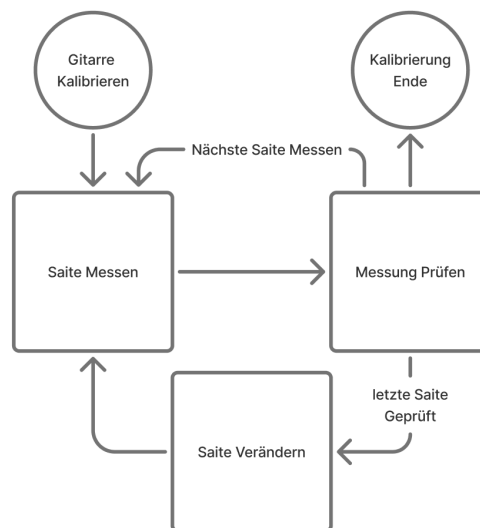


Abbildung 33: Interaktionsdesign – Kalibrierung

Der Kalibrierungsvorgang nutzt drei Seiten, die in einem festgelegten Ablauf durchlaufen werden.

Seite 1 – Saite messen und **Seite 2 – Messung prüfen** wechseln sich ab, bis alle sechs Saiten gemessen sind. So wird der Ausgangszustand der Gitarre als erster Messpunkt erfasst (FSA 7).

Anschließend folgt **Seite 3 – Saite verändern**: Der Nutzer verstimmt eine bestimmte Saite. Daraufhin wechseln sich Seite 1 und Seite 2 erneut ab, bis alle sechs Saiten gemessen sind und der Einfluss der Verstimmung auf die übrigen Saiten erfasst ist. Der zuletzt gemessene Zustand dient dabei jeweils als

Ausgangslage für die folgende Verstimmung. Dieser Vorgang wiederholt sich für jede der sechs Saiten (FSA 7, FSA 9).

Sobald alle Messpunkte erfasst sind, kann der Nutzer sämtliche Messdaten in einer Gesamtübersicht einsehen und fehlerhafte Einträge korrigieren (FBA 8), bevor die Kalibrierung abgeschlossen wird.

5.2.2.3 Stimmen

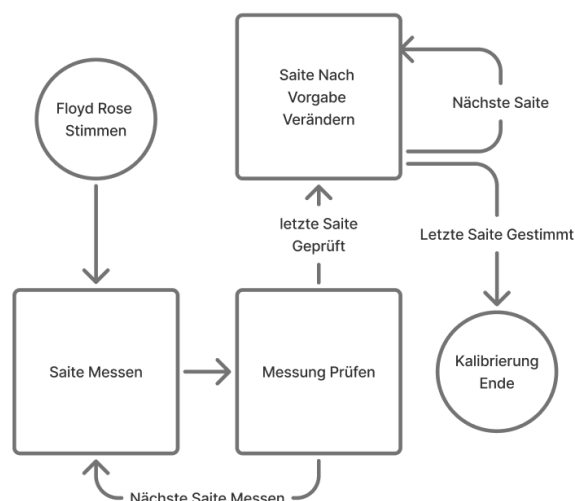


Abbildung 34: Interaktionsdesign – Stimmprozess

Der Stimmprozess folgt einem ähnlichen Ablauf wie die Kalibrierung. Zunächst spielt der Nutzer jede Saite einzeln an; nach jeder Messung kann er den erfassten Wert auf einer Bestätigungsseite prüfen. Sobald alle sechs Saiten gemessen sind, berechnet das System die individuellen Zielfrequenzen gemäß FSA 3 und führt den Nutzer saitenweise durch den Stimmvorgang, bis die Gitarre korrekt gestimmt ist (FBA 1).

5.2.3 Visuelles Konzept

Da die App in der Entwicklungsphase primär auf Android getestet wird, werden die Gestaltungsrichtlinien von Google – das Material-3-Designsystem [18] – als visuelle Grundlage verwendet. Material 3 definiert verbindliche Standards für Typografie, Farbschemata, Icons, Interaktionszustände von Komponenten sowie Navigationsstrukturen und gewährleistet damit eine plattformkonforme, konsistente Darstellung (NFA-KO 3).

Für Konzeption und UI-Design wird Figma³ eingesetzt, das ebenfalls ein offizielles Material-3-Komponentenset bereitstellt und so einen konsistenten Übergang vom Entwurf zur Implementierung ermöglicht.

5.2.4 Prototypen

Die App wurde zunächst als vertikaler Prototyp entwickelt, um frühzeitig zu überprüfen, ob FBA 1 – das erfolgreiche Stimmen einer Floyd-Rose-Gitarre – grundsätzlich umsetzbar ist. Im Anschluss wurde die Benutzeroberfläche iterativ überarbeitet, um die App auch für weniger technisch versierte Nutzer zugänglich zu machen (NFA-BE 1).

Abbildung 35 zeigt einen frühen Prototyp der Kalibrierungsseite, der trotz technischer Funktionsfähigkeit eine hohe Informationsdichte aufwies und Testpersonen überforderte. In Abbildung 36 ist die

³<https://www.figma.com/>

überarbeitete Version zu sehen, in der die Oberfläche deutlich vereinfacht und die Nutzerführung strukturiert wurde.

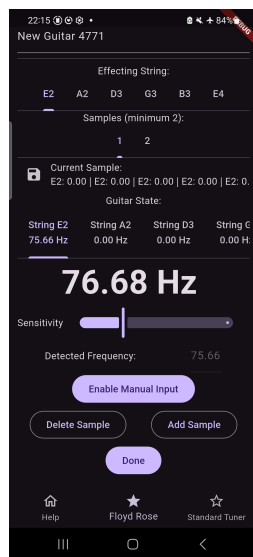


Abbildung 35: Früher Prototyp: Messung der Verstimmungsmatrix

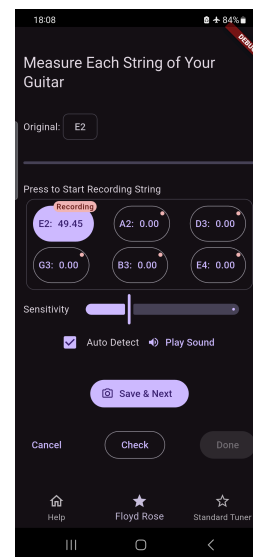


Abbildung 36: Überarbeiteter Prototyp: Kalibrierungsansicht

5.2.5 Wireframes

Auf Basis der Informationsarchitektur und des Interaktionsdesigns wurden Wireframes erstellt, um die Benutzeroberfläche zu konkretisieren und optimieren (Abbildung 37 bis Abbildung 44).

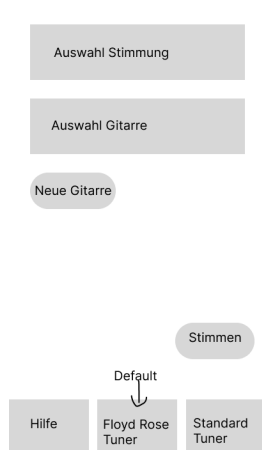


Abbildung 37: Wireframe: Startseite

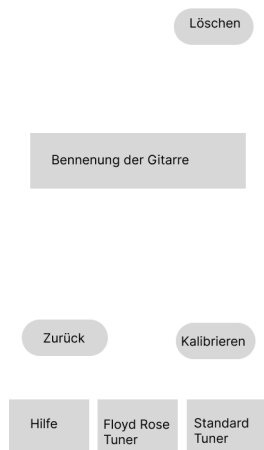


Abbildung 38: Wireframe: Gitarre bearbeiten



Abbildung 39: Wireframe: Kalibrierung – Saite messen

Abbildung 37 zeigt die Startseite, deren Struktur der Informationsarchitektur aus Abbildung 31 folgt. Über eine Bottom-Tab-Navigation sind die Hilfsseite und das Standard-Stimmgerät jederzeit erreichbar. Auf der Startseite selbst kann der Nutzer eine Zielstimmung sowie ein Gitarrenprofil auswählen. Gemäß dem Nutzungsszenario von Jonas (Abschnitt 5.1.5.3) ist zudem das direkte Anlegen einer neuen Gitarre prominent platziert. Ein großer Stimmen-Button leitet den Floyd-Rose-Stimmprozess ein (FBA 1).

Abbildung 38 zeigt die Bearbeitungsansicht einer gespeicherten Gitarre. Von hier aus kann der Nutzer das Profil umbenennen (FBA 12), löschen sowie die Kalibrierung starten (FBA 11). Abbildung 39 zeigt

den ersten Schritt der Kalibrierung. Am oberen Rand informiert ein Fortschrittsindikator über den aktuellen Stand. Darunter wird der Nutzer aufgefordert, eine bestimmte Saite anzupspielen; die laufende Frequenzmessung wird unmittelbar darunter visualisiert. Ein Weiter-Button führt zu Abbildung 40.



Abbildung 40: Wireframe: Kalibrierung – Messung prüfen



Abbildung 41: Wireframe: Kalibrierung – Saite verstimmen



Abbildung 42: Wireframe: Kalibrierung – Gesamtübersicht

Dort kann der Nutzer die erfasste Messung überprüfen. Anstelle einer numerischen Frequenzanzeige – die in den Nutzertests zu Verwirrung führte (Abschnitt 6.3, Nutzer 3) – ist ein Button vorgesehen, der einen Ton in der gemessenen Frequenz abspielt. So kann der Nutzer auditiv beurteilen, ob die richtige Frequenz erkannt wurde, ohne Zahlenwerte interpretieren zu müssen (FBA 7). Bei einer fehlerhaften Messung kann er den Schritt wiederholen. Sind alle Saiten gemessen, führt das Interaktionsdesign (Abbildung 33) zur nächsten Phase: Abbildung 41 fordert den Nutzer auf, eine bestimmte Saite zu verstimmen. Ein visuelles Feedback-Element soll dabei bestätigen, dass die Verstimmung erkannt wurde (FSA 9).

Nach Abschluss aller Messreihen gelangt der Nutzer zu Abbildung 42, einer Gesamtübersicht aller erfassten Messwerte. Von hier aus können einzelne Messungen gezielt korrigiert und die entsprechende Stelle im Kalibrierungsablauf direkt angesprungen werden (FBA 8).

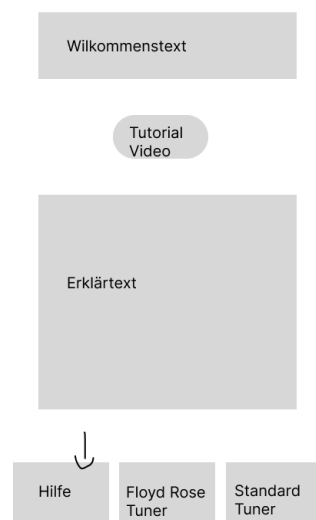


Abbildung 43: Wireframe: Hilfsseite

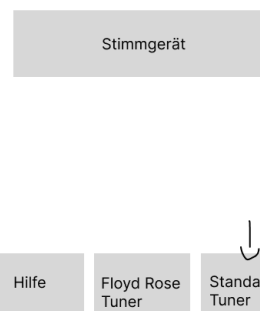


Abbildung 44: Wireframe: Standard-Stimmgerät

Abbildung 43 und Abbildung 44 zeigen den Aufbau der Hilfsseite (FBA 6, NFA-BE 4) sowie des Standard-Stimmgeräts (FSA 11, FBA 10), das unabhängig vom Floyd-Rose-Algorithmus zur schnellen Überprüfung der Stimmung genutzt werden kann.

5.2.6 Design

5.2.6.1 Onboarding

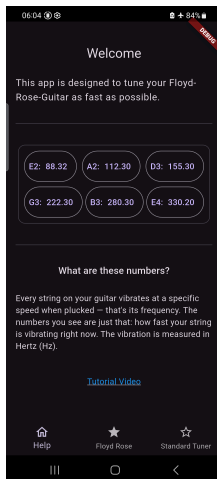


Abbildung 45: Design: Hilfsseite

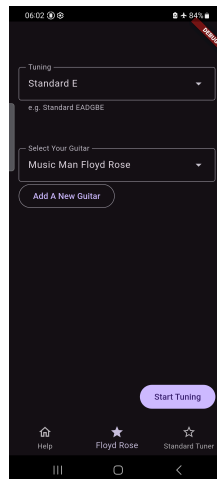


Abbildung 46: Design: Startseite

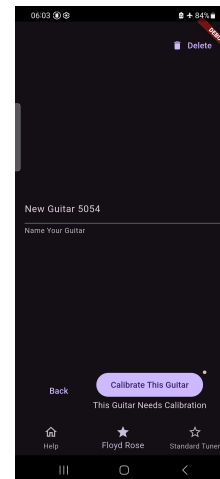


Abbildung 47: Design: Gitarre bearbeiten

Beim ersten Start der App wird der Nutzer auf die Hilfsseite (Abbildung 45) geleitet, die eine Übersicht der App-Funktionen sowie einen Link zu einem Tutorial-Video enthält (FBA 6, NFA-BE 4). Von dort gelangt er zur Startseite (Abbildung 46), die bei allen folgenden Starts direkt angezeigt wird. Bereits gespeicherte Gitarrenprofile und Stimmungen sind vorausgewählt; ein prominenter „Start Tuning“-Button leitet den Stimmvorgang unmittelbar ein (FBA 1).

Über „Add A New Guitar,“ legt der Nutzer ein neues Gitarrenprofil an und gelangt zur Gitarrenansicht (Abbildung 47). Dort wird vorab ein Name mit zufälliger ID generiert, der frei angepasst werden kann (FBA 12). Der „Calibrate This Guitar“-Button ist mit einem Badge versehen und zieht damit die Aufmerksamkeit des Nutzers auf sich; der begleitende Hinweistext „This Guitar Needs Calibration,“ macht den nächsten Schritt unmissverständlich deutlich (NFA-BE 1). Ein Klick auf den Button startet sofort den Kalibrierungsvorgang.

5.2.6.2 Kalibrierung

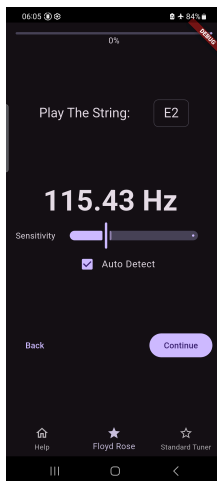


Abbildung 48: Design: Kalibrierung – Saite messen

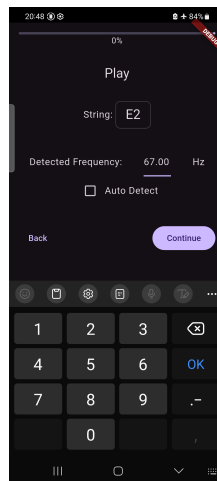


Abbildung 49: Design: Manuelle Frequenzeingabe

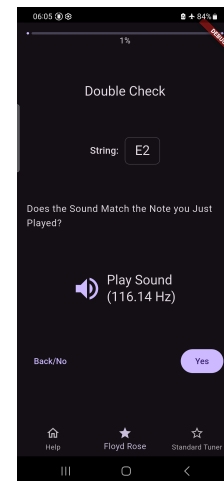


Abbildung 50: Design: Kalibrierung – Messung prüfen

Abbildung 48 zeigt die Messseite. Ein Empfindlichkeitsregler mit Live-Lautstärkeanzeige erlaubt die Anpassung des Mikrofonschwellenwerts (FBA 5). Eine Checkbox „Auto Detect,“ steuert den Erkennungsmodus: Wird sie deaktiviert, erscheint ein Textfeld, in das der Nutzer eine extern gemessene Frequenz manuell eintragen kann (Abbildung 49, FBA 14). Sobald eine Frequenz erkannt oder eingetragen wurde, führt „Continue“ zur Prüfseite.

Auf der Prüfseite (Abbildung 50) befindet sich zentral und gut erreichbar ein „Play Sound,“-Button, der die erkannte Frequenz als Ton abspielt. Der begleitende Text fragt den Nutzer, ob der abgespielte Ton mit dem zuvor gespielten übereinstimmt. Bei Nein kehrt er zur Messseite zurück und wiederholt die Aufnahme; bei Ja wird die nächste Saite gemessen (FBA 7, FBA 8).

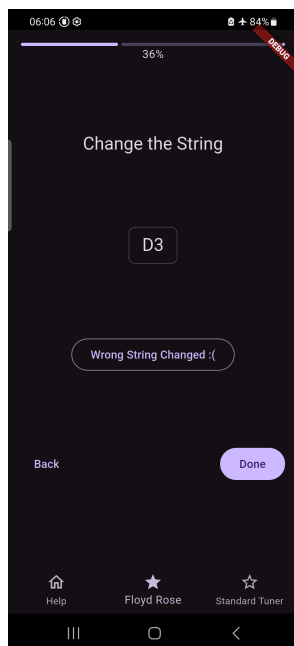


Abbildung 51: Design: Kalibrierung – Saite verstimmen

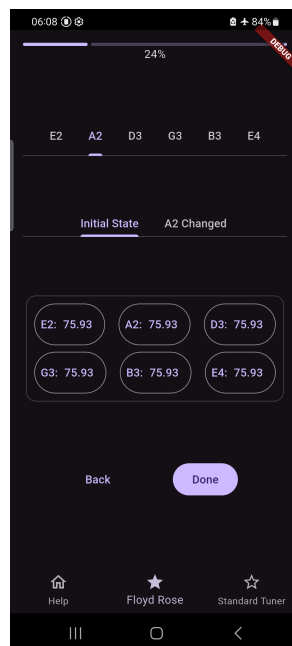


Abbildung 52: Design: Kalibrierung – Gesamtübersicht

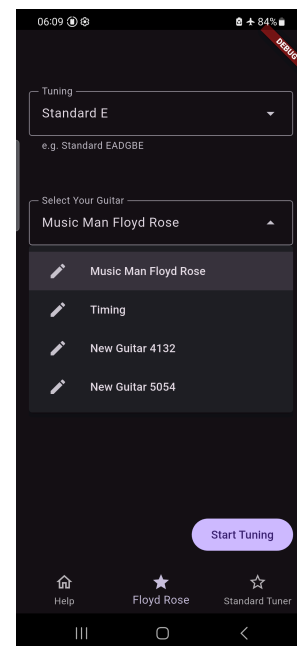


Abbildung 53: Design: Gitarre auswählen

Abbildung 51 fordert den Nutzer auf, eine bestimmte Saite zu verstimmen. Für den Fall, dass versehentlich die falsche Saite verändert wurde, steht ein „Wrong String Changed“-Button zur Verfügung, der den Schritt zurücksetzt (FSA 9). Nach dem Verstimmen bestätigt der Nutzer mit „Done“ und der Ablauf folgt dem Interaktionsdesign aus Abbildung 33.

Nach Abschluss aller Messreihen gelangt der Nutzer zur Gesamtübersicht (Abbildung 52). Die obere Tab-Navigation zeigt die Saite, deren Verstimmung das jeweilige Sample beschreibt; die untere Tab-Navigation wechselt zwischen den einzelnen Messpunkten. Jedes Sample besteht aus sechs Einträgen – einem Frequenzwert pro Saite – die auf zwei Nachkommastellen gerundet angezeigt werden. Über „Done“, wird die Kalibrierung abgeschlossen und der Nutzer kehrt zur Startseite zurück (Abbildung 46, FBA 8).

Möchte der Nutzer auf der Startseite (Abbildung 46) eine gespeicherte Gitarre auswählen, öffnet sich eine Auswahlliste (Abbildung 53). Jeder Eintrag enthält ein Stift-Icon, über das der Nutzer direkt zur Gitarrenansicht navigiert. Wurde eine Gitarre bereits kalibriert, zeigt Abbildung 54, dass der Kalibrierungsbutton weiterhin vorhanden, aber weniger prominent gestaltet ist, um erfahrene Nutzer nicht zu bremsen (FBA 11).

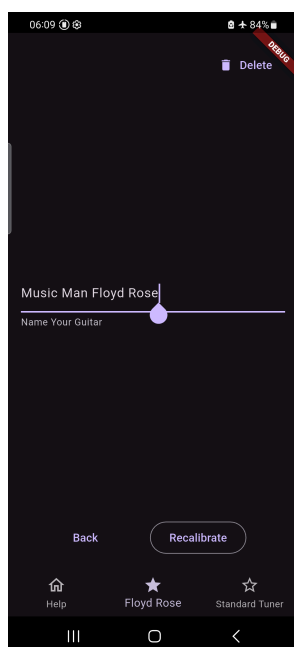


Abbildung 54: Design: Rekali-
brieren

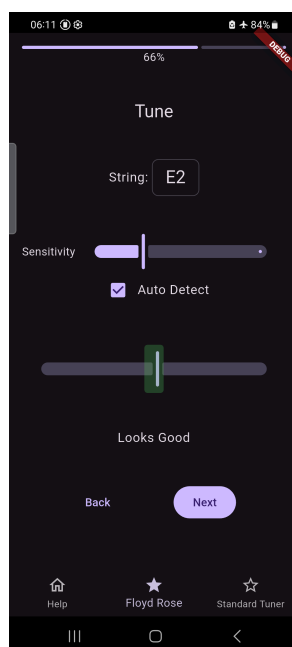


Abbildung 55: Design: Floyd-
Rose-Tuner

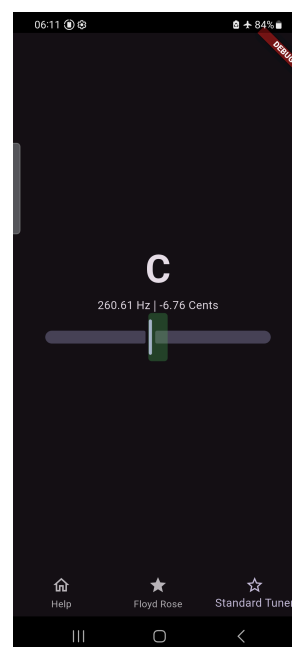


Abbildung 56: Design: Stan-
dard-Stimmgerät

5.2.6.3 Stimmvorgang

Ein Klick auf „Start Tuning“, führt zunächst durch dieselben Mess- und Prüfseiten wie die Kalibrierung (Abbildung 48, Abbildung 50), um den aktuellen Zustand der Gitarre zu erfassen. Nach dem Messen aller sechs Saiten wird der Nutzer zur Floyd-Rose-Tuner-Seite weitergeleitet (Abbildung 55). Dort spielt er die angezeigte Saite an und stimmt sie, bis der Slider in den grünen Zielbereich rückt. „Next“ wechselt zur nächsten Saite (FBA 1, FSA 3, FSA 4).

Nach dem Stimmen aller Saiten gelangt der Nutzer zum Standard-Stimmgerät (Abbildung 56), das die gespielte Note, die Frequenz in Hertz sowie die Abweichung in Cent anzeigt und so eine abschließende Überprüfung der Stimmung ermöglicht (FSA 11, FBA 10).

5.3 Architektur

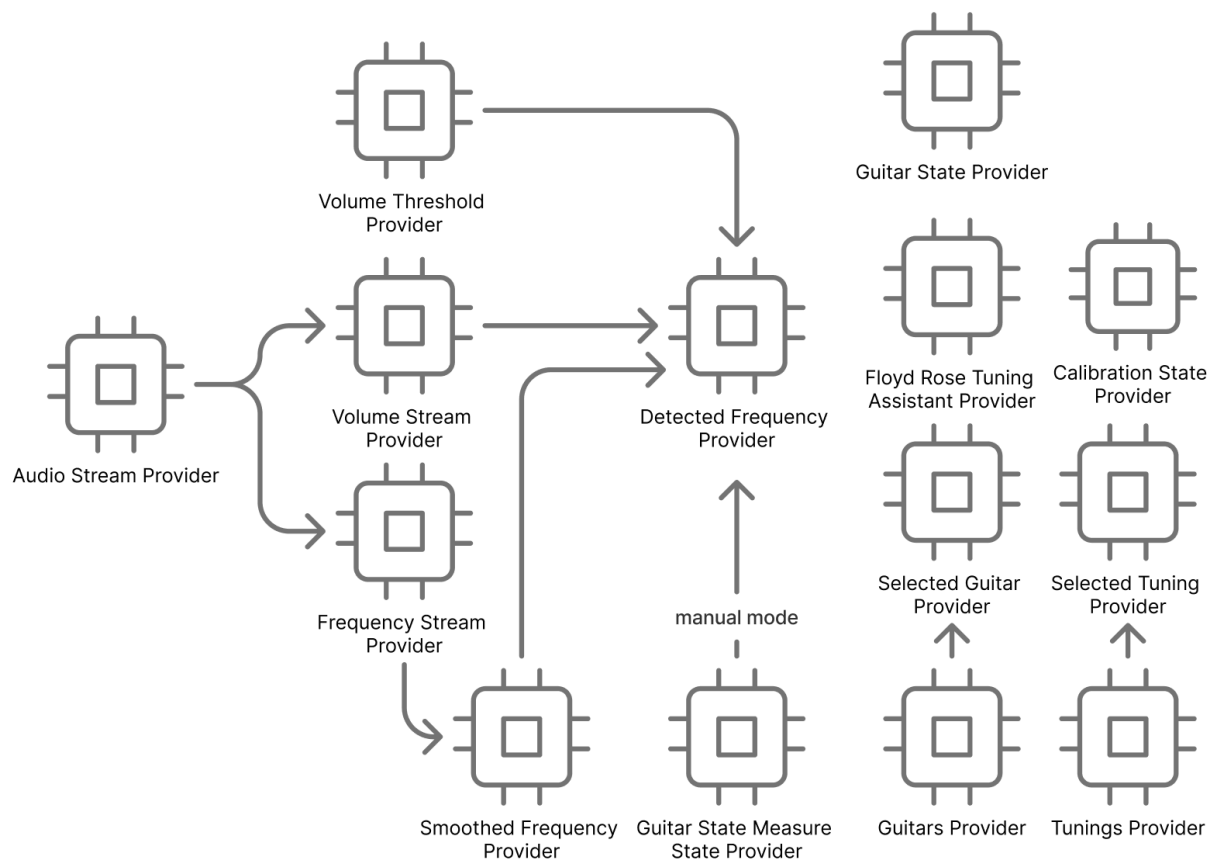


Abbildung 57: Architektur der App

Die App folgt dem Architekturmuster *Model-View-ViewModel* (MVVM). Dieses Muster eignet sich besonders, weil das ViewModel die Schnittstelle zwischen View und Model kapselt: Die UI kennt keine Implementierungsdetails des Modells und das Modell ist unabhängig von der Darstellung. In Verbindung mit Riverpod (Abschnitt 5.4.2) wird dieses Muster durch sogenannte *Provider* und *Notifier* umgesetzt – Provider stellen lesenden Zugriff auf den Zustand bereit, Notifier ermöglichen dessen Veränderung und benachrichtigen die UI automatisch über Änderungen, sodass betroffene Widgets neu gerendert werden. Ein wesentlicher Vorteil dieses Ansatzes ist, dass kein manuelles Prop-Passing durch den gesamten Widget-Baum erforderlich ist.

5.3.1 Audio Stream Provider

Der Audio Stream Provider stellt einen kontinuierlichen Datenstrom der Mikrofonaufnahme bereit. Der Datentyp ist ein `Stream<Int16List>`, der die rohen PCM-Samples des Mikrofons als 16-Bit-Integer ausgibt und von allen nachgelagerten Signalverarbeitungs-Providern konsumiert wird (FSA 1).

5.3.2 Volume Stream Provider

Abonniert den Audio Stream Provider (Abschnitt 5.3.1) und berechnet den Schalldruckpegel des Eingangssignals in dBFS gemäß FSA 2. Der Ausgabewert ist ein `Stream<double>` im Wertebereich $[-60.0, 0.0] \sim \text{dBFS}$.

5.3.3 Frequency Stream Provider

Abonniert den Audio Stream Provider (Abschnitt 5.3.1) und wendet den YIN-Algorithmus [9] auf den Audiodatenstrom an (FSA 1). Die Ausgabe ist ein `Stream<double>`, der die geschätzte Grundfrequenz in Hertz liefert.

5.3.4 Smoothed Frequency Provider

Abonniert den Frequency Stream Provider (Abschnitt 5.3.3) und glättet die eingehenden Messwerte durch einen gleitenden Mittelwert (FSA 5), um kurzfristige Ausreißer zu dämpfen. Die Ausgabe ist ein `Stream<double>`.

5.3.5 Volume Threshold Provider

Liefert den konfigurierbaren Lautstärkeschwellenwert im Wertebereich $[-60.0, 0.0] \sim \text{dBFS}$ (FSA 2). Der Wert wird über den Empfindlichkeits-Schieberegler in den Einstellungen gesteuert (FBA 5).

5.3.6 Detected Frequency Provider

Kombiniert die Ausgaben des Smoothed Frequency Providers, des Volume Stream Providers (Abschnitt 5.3.2) und des Volume Threshold Providers und liefert die zuletzt erkannte Frequenz als `double`.

Ist manuelle Erkennung aktiviert (`manualDetection == true`), wird die Frequenz ausschließlich über einen Notifier gesetzt – etwa durch eine Texteingabe (FBA 14). Andernfalls wird der aktuelle Wert des Smoothed Frequency Providers übernommen, sofern der Schalldruckpegel zum selben Zeitpunkt den Schwellenwert überschreitet (FSA 2).

5.3.7 String Measure State Provider

Stellt das `StringMeasureState`-Objekt bereit, das den aktuellen Fortschritt der saitenweisen Messung abbildet:

```
StringMeasureState {  
    int currentStringIndex    // Index der aktuell zu messenden Saite (0–5)  
    bool manualDetection      // Eingabemodus: automatisch oder manuell  
}
```

5.3.8 Guitar State Provider

Repräsentiert den aktuellen Frequenzzustand aller sechs Saiten als `List<double>` der Länge 6. Jeder Eintrag entspricht der zuletzt gemessenen Frequenz der jeweiligen Saite. Dieser Provider bildet das zentrale zustandstragende Modell während des Stimm- und Kalibrierungsvorgangs und kann von beliebigen Widgets – etwa über Button-Interaktionen – aktualisiert werden.

5.3.9 Calibration State Provider

Stellt das `CalibrationState`-Objekt bereit, das den Fortschritt innerhalb der Kalibrierung verfolgt (FSA 7):

```
CalibrationState {  
    int currentEffectingStringIndex // Index der aktuell verstimmten Saite  
    int currentSampleIndex          // Index des aktuellen Messpunkts  
}
```

5.3.10 Guitars- und Selected-Guitar-Provider

Der Guitars-Provider verwaltet die persistente Speicherung aller Guitar-Objekte auf dem Gerät (FSA 10, FBA 3). Der Selected-Guitar-Provider hält das aktuell ausgewählte Profil vor.

Ein Guitar-Objekt ist wie folgt aufgebaut:

```
Guitar {  
    String guitarName           // Benutzerdefinierter Name (@req-fsa-13)  
    List<List<double>> matrix   // Verstimmungsmatrix C  
    List<List<double>> inverseMatrix // Gecachte Inverse von C  
    Map<int, List<GuitarState>> samples // Rohmessdaten der Kalibrierung  
}
```

Guitar.samples speichert alle Messpunkte, aus denen die Verstimmungsmatrix berechnet wird (FSA 7). Der erste Schlüssel bezeichnet die verstimmte Saite, der zweite den Messpunkt-Index. So entspricht samples[1][2] dem dritten erfassten Gitarrenzustand, als Saite A2 (Index 1) verstimmt wurde – also einem Frequenzvektor der Form $[f_{E2}, f_{A2}, f_{D3}, f_{G3}, f_{B3}, f_{E4}]$.

matrix und inverseMatrix lassen sich vollständig aus samples ableiten, werden jedoch aus Performancegründen gecacht und gemeinsam persistiert.

5.3.11 Tunings- und Selected-Tuning-Provider

Der Tunings-Provider verwaltet die persistente Speicherung aller Tuning-Objekte (FSA 14, FSA 15). Der Selected-Tuning-Provider hält die aktuell gewählte Stimmung vor.

Ein Tuning-Objekt ist wie folgt aufgebaut:

```
Tuning {  
    String name           // Bezeichnung der Stimmung  
    List<String> goalNotes // Zielnoten im Format "E2", "A2", ...  
}
```

Tuning.goalNotes enthält die standardisierten Notenbezeichnungen im Format *Note + Oktave* (z. B. "E2" für das E in der zweiten Oktave) und definiert damit die Zielfrequenzen für den Stimmprozess (FSA 3).

5.4 Implementierung

5.4.1 Auswahl des Cross-Platform-Frameworks

Da die App gemäß NFA-KO 1 auf iOS und Android lauffähig sein muss, wird ein Cross-Platform-Framework eingesetzt, um eine gemeinsame Codebasis für beide Plattformen zu erhalten. Zur Auswahl standen React Native [19], .NET MAUI [20] und Flutter [21].

5.4.1.1 Umsetzungsgeschwindigkeit komplexer Anforderungen

Flutter bietet mit seinem Widget-System und dem integrierten Rendering-Stack eine durchgängige Abstraktionsebene, auf der sich komplexe UI-Zustände – wie die Echtzeit-Visualisierung der Stimmgenauigkeit – direkt und ohne Umwege über plattformspezifische APIs umsetzen lassen. React Native erfordert bei komplexeren Anforderungen häufiger den Rückgriff auf native Module oder externe Bibliotheken, was den Entwicklungsaufwand erhöht. .NET MAUI zeigte in eigenen Experimenten bei plattformübergreifenden UI-Komponenten Inkonsistenzen, die zusätzlichen Abstimmungsaufwand erzeugten.

5.4.1.2 Einstiegshürde

Flutter verwendet die Programmiersprache Dart, die im Vergleich zu JavaScript (React Native) oder C# (.NET MAUI) zunächst eine unbekanntere Wahl darstellt. In der Praxis erwies sich Dart jedoch als schnell erlernbar: Die Sprache ist stark typisiert, die Dokumentation vollständig und das Tooling – insbesondere Hot Reload und die IDE-Integration – reduziert die Zeit bis zu ersten funktionierenden

Ergebnissen erheblich. Aus eigener Erfahrung war die Einstiegshürde bei Flutter trotz der neuen Sprache geringer als bei .NET MAUI, dessen Abstraktion über native Controls zu unerwartetem Verhalten und schwer nachvollziehbaren Fehlern führte.

5.4.1.3 Codestabilität

Flutter kompiliert Ahead-of-Time (AOT) in nativen Code und umgeht damit die Bridge-Architektur von React Native, die bei häufigem Datenaustausch zwischen JavaScript- und nativer Ebene Laufzeitfehler und Performance-Einbrüche verursachen kann. Da die App gemäß FSA 1 kontinuierliche Frequenzmessungen in Echtzeit durchführt, ist ein stabiler, vorhersehbarer Ausführungspfad ohne Kommunikations-Overhead essenziell. Darüber hinaus reduziert Flutters geschlossenes Ökosystem – im Gegensatz zu Reacts starker Abhängigkeit von externen Bibliotheken – das Risiko inkompatibler Abhängigkeiten und erleichtert langfristige Wartung.

5.4.1.4 UI-Komponenten mit Material 3

Flutter integriert Material 3 [18] direkt als Teil des Frameworks, ohne externe Abhängigkeit. Standardkomponenten wie Buttons, Navigation, Eingabefelder und Farbschemata stehen sofort zur Verfügung und folgen konsistenten Designrichtlinien. Für diese App bedeutet das, dass die Benutzeroberfläche – insbesondere die geführte Kalibrierung und die Stimmansicht gemäß NFA-BE 1 und NFA-BE 2 – ohne eigenes Design-System von Grund auf aufgebaut werden musste. Theming, Typografie und Abstände sind systemseitig definiert und plattformweit konsistent, was den Designaufwand erheblich reduziert und gleichzeitig eine professionelle Darstellung sicherstellt.

5.4.1.5 Entscheidung

Aufgrund eigener praktischer Erfahrungen mit allen drei Frameworks sowie der beschriebenen Eigenschaften wurde Flutter gewählt. Es bietet bessere Performance als React Native, ein kohärenteres Entwicklungserlebnis als .NET MAUI und ermöglicht eine plattformübergreifende Darstellung, die auf jedem Gerät identisch aussieht. Nachteile von Cross-Compiling-Ansätzen, die in [15] (2017) beschrieben werden, sind durch Flutters AOT-Kompilierung, eigenständiges Rendering und die direkte Integration von Material 3 heute weitgehend überholt.

5.4.2 Abhängigkeiten

Die App verwendet folgende Laufzeit-Abhängigkeiten:

Paket	Version	Verwendung
flutter_riverpod	^3.0.0	State-Management; stellt gemeinsamen Zustand über den Widget-Baum bereit und bildet die Grundlage für Provider und Notifier.
riverpod_annotation	^4.0.0	Annotationspaket für Riverpod; wird zur Code-Generierung benötigt.
shared_preferences	^2.3.3	Persistenz einfacher Schlüssel-Wert-Paare; speichert Gitarrenprofile und Stimmungen dauerhaft auf dem Gerät (FSA 10, FSA 13).
json_annotation	^4.9.0	Annotationspaket für JSON-Serialisierung; ermöglicht das Speichern komplexer Objekte als JSON-String in shared_preferences.

Paket	Version	Verwendung
record	^6.1.1	Plattformunabhängiger Zugriff auf das Gerätmikrofon für iOS und Android (FSA 1).
pitch_detector_dart	^0.0.7	Implementierung des YIN-Algorithmus zur Grundfrequenzschätzung (FSA 1).
buffered_list_stream	^1.3.0	Puffert den kontinuierlichen Audiodatenstrom zu Blöcken, die der YIN-Algorithmus verarbeiten kann.
ml_linalg	^13.12.6	Optimierte Matrizenoperationen (Multiplikation, Inversion) für Verstimmungsmatrix und Zielfrequenzberechnung (FSA 7, FSA 3).
statistics	^1.2.1	Hilfsfunktionen für Listenoperationen in der Signalverarbeitung und Plausibilitätsprüfung.
async	^2.13.0	Erweiterungen für asynchrone Programmierung und Parallelverarbeitung des Audiostreams.
flutter_sound	^9.30.0	Wiedergabe eines Referenztons zur auditiven Überprüfung der Stimmung.
auto_route	^11.1.0	Typsicheres Routing mit reduziertem Boilerplate-Code.
url_launcher	^6.3.2	Öffnet externe URLs; verlinkt Hilfevideos auf YouTube (FBA 6, NFA-BE 4).

Folgende Abhängigkeiten werden ausschließlich zur Entwicklungszeit benötigt und sind nicht Teil des ausgelieferten Artefakts:

Paket	Version	Verwendung
riverpod_generator	^4.0.0+1	Generiert Riverpod-Provider aus Annotationen.
riverpod_lint	^3.0.0	Statische Analyse für korrekte Riverpod-Verwendung.
json_serializable	^6.11.1	Generiert JSON-Serialisierungscode aus json_annotation-Annotationen.
auto_route_generator	^10.2.4	Generiert Routing-Code aus auto_route-Annotationen.
build_runner	^2.7.1	Führt alle Code-Generatoren aus.
flutter_lints	^6.0.0	Offizielles Lint-Regelwerk für Flutter; stellt statische Codequalität sicher.

5.4.3 Quellcode

Die App wurde mit Git versioniert und ist öffentlich auf GitHub verfügbar: https://github.com/Raphael2b3/floyd_rose_tuner

5.4.4 Kalibrierungslogik

Um die Fortschrittsanzeige und eine Zurück-Funktion zu implementieren, muss zu jedem Zeitpunkt eindeutig bestimmbar sein, in welchem Schritt des Kalibrierungsvorgangs sich die App befindet und wie der Zustand bei „Weiter,“ bzw. „Zurück“ zu setzen ist.

Der Kalibrierungsablauf wird durch vier Variablen vollständig beschrieben:

Variable	Wertebereich	Bedeutung
pageIndex	$\in \mathbb{N}, [0, 2]$	Aktuelle Seite: 0 = Messen, 1 = Prüfen, 2 = Verändern
stringIndex	$\in \mathbb{N}, [0, 5]$	Index der Saite, mit der aktuell interagiert wird
sampleIndex	$\in \mathbb{N}, [0, N - 1]$	Index des aktuellen Messpunkts
effectIndex	$\in \mathbb{N}, [0, 5]$	Index der Saite, deren Verstimmung das Sample beschreibt

Es wird die minimale Anzahl der benötigten Messpunkte verwendet: $N = 2$.

5.4.4.1 Fortschrittsberechnung

Aus diesen Variablen lässt sich ein linearer Fortschrittswert ableiten:

$$\text{progress} = \text{pageIndex} + 2 \cdot \text{stringIndex} + 13 \cdot \text{sampleIndex} + 26 \cdot \text{effectIndex} \quad (37)$$

Der maximale Wert ergibt sich zu:

$$\text{max} = 2 + 2 \cdot 5 + 13 \cdot 2 + 26 \cdot 5 = 168 \quad (38)$$

Der normierte Fortschritt $p = \text{progress} / \text{max} \in [0, 1]$ wird direkt als Füllstand der Fortschrittsanzeige verwendet.

5.4.4.2 Navigationslogik

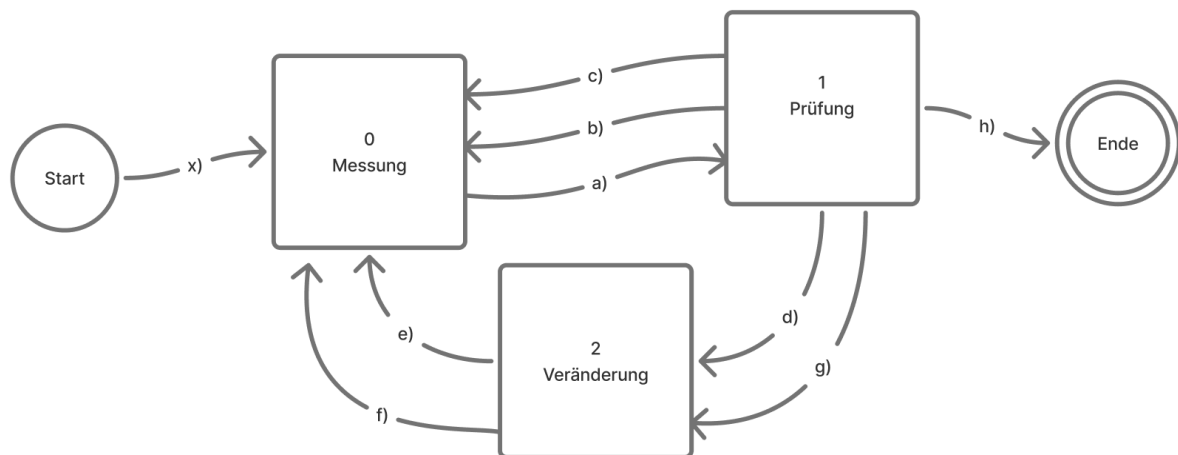


Abbildung 58: Zustandsdiagramm: Kalibrierungslogik

Die Zustandsübergänge beim Drücken von „Weiter“, sind in Abbildung 58 mit den Labels a) bis h) dargestellt und werden im Folgenden als Pseudocode ausformuliert.⁴

Um Übergang c) und b) auf der Rückwärtsnavigation unterscheiden zu können, wird die Hilfsvariable `prüfungFehler` eingeführt. Sie ist `true`, wenn der Nutzer auf der Prüfseite „Zurück“, gedrückt und damit Übergang c) ausgelöst hat; andernfalls ist sie `false`.

⁴++ ist die Kurzform für $v = v + 1$; analog gilt -- für $v = v - 1$.

Messseite

```
Weiter-Button{ // Übergang a)
    navigate(Prüfseite);
}

Zurück-Button{
    if stringIndex == 0 && sampleIndex == 0 && effectIndex == 0:
        // Übergang x)
        exit()
    else if stringIndex > 0: // Übergang b)
        if not prüfungFehler:
            stringIndex--
            // Übergang c) → keine Zustandsänderung
            navigate(Prüfseite)
    else: // stringIndex == 0
        stringIndex = 5
        if sampleIndex > 0: // Übergang e)
            sampleIndex--
        else: // Übergang f)
            sampleIndex = 1 // letztes Sample der vorherigen Verstimmung
            navigate(Veränderungsseite)
}
```

Prüfseite

```
Weiter-Button{
    samples[effectIndex][sampleIndex][stringIndex] = messung

    if stringIndex < 5: // Übergang b)
        stringIndex++
        navigate(Messseite)

    else if sampleIndex == 1 && effectIndex == 5:
        calculateMatrix() // Übergang h)
        saveGuitar()
        navigate(Kontrollansicht)

    else:
        if sampleIndex == 0: // Übergang d)
            stringIndex = effectIndex
        else: // Übergang g) sampleIndex==1 && effectIndex<5
            samples[effectIndex + 1][0] = samples[effectIndex][1]
            // Ausgangslage für nächste Verstimmung übernehmen
            effectIndex++
            stringIndex = effectIndex
            sampleIndex = 0
            navigate(Veränderungsseite)
}

Zurück-Button{ // Übergang c)
    prüfungFehler = true
    navigate(Messseite)
}
```

Veränderungsseite

```

Fertig-Button{ // Übergang e)
    sampleIndex++
    stringIndex = 0
    navigate(Messseite)
}

WrongStringChanged-Button{ // Übergang f)
    sampleIndex = 0
    stringIndex = 0
    navigate(Messseite)
}

Zurück-Button{ // Übergang d) rückwärts
    stringIndex = 5
    if sampleIndex == 1 && effectIndex > 0: // Übergang g) rückwärts
        effectIndex--
    navigate(Prüfseite)
}

```

5.4.5 Stimmlogik

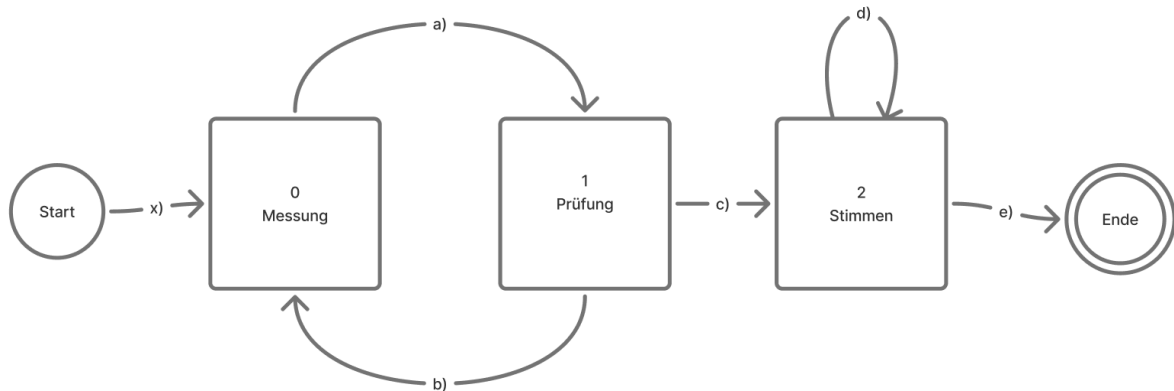


Abbildung 59: Zustandsdiagramm: Stimmlogik

Der Stimmvorgang wird durch zwei Variablen beschrieben:

Variable	Wertebereich	Bedeutung
pageId	$\in \mathbb{N}, \{0, 1, 2\}$	Aktuelle Seite: 0 = Messen, 1 = Prüfen, 2 = Stimmen
stringIndex	$\in \mathbb{N}, [0, 5]$	Index der Saite, mit der aktuell interagiert wird

5.4.5.1 Fortschrittsberechnung

Da die Stimmseite eine abweichende Gewichtung benötigt, ist der Fortschrittswert stückweise definiert:

$$\text{progress} = \begin{cases} \text{pageId} + 2 \cdot \text{stringIndex} & \text{pageId} \neq 2 \\ 12 + \text{stringIndex} & \text{pageId} = 2 \end{cases} \quad \text{max} = 18 \quad (39)$$

5.4.5.2 Navigationslogik

Die Zustandsübergänge beim Drücken von „Weiter„ und „Zurück“ sind in Abbildung 59 mit den Labels a) bis f) dargestellt und werden im Folgenden als Pseudocode ausformuliert.

Messeite

```
Weiter-Button:{ // Übergang a)
    navigate(Prüfseite)
}

Zurück-Button{
    if stringIndex == 0: // Übergang x)
        exit()
    else: // Übergang b)
        stringIndex--
        navigate(Prüfseite)
}
```

Prüfseite

```
Weiter-Button{
    speicherMessung(stringMeasureState[stringIndex] = messung)

    if stringIndex < 5: // Übergang b)
        stringIndex++
        navigate(Messeite)
    else: // Übergang c)
        stringIndex = 0
        berechneZielfrequenzen()
        navigate(Stimmseite)
}

Zurück-Button{ // reverse a)
    prüfungFehler=true
    navigate(Messeite)
}
```

Stimmseite

```
Weiter-Button{
    if stringIndex < 5: // Übergang d)
        stringIndex++
    else: // Übergang e)
        navigate(Ende)
        return
    navigate(Stimmseite) // gleiche Seite, neue Saite
}

Zurück-Button{
    if stringIndex > 0: // Übergang d) rückwärts
        stringIndex--
    else: // Übergang e) rückwärts
        stringIndex = 5
        navigate(Prüfseite)
        return
    navigate(Stimmseite)
}
```

5.5 Tests während der Entwicklung

5.5.1 Manuelle Tests

Aus Zeitgründen wurden die meisten Anforderungen manuell getestet. Dabei wurde jede Anforderung durch gezielte Bedienung der App unter den jeweils beschriebenen Bedingungen überprüft und das Ergebnis protokolliert.

Manuell getestet wurden:

- FBA 1 bis FBA 16
- FSA 1, FSA 2
- FSA 4 bis FSA 6
- FSA 9 bis FSA 16
- NFA-MG 1 bis NFA-MG 3
- NFA-RO 2, NFA-RO 3
- NFA-BE 1 bis NFA-BE 4
- NFA-DP 1 bis NFA-DP 3

5.5.2 Unit-Tests

Für die mathematisch verifizierbaren Kernanforderungen wurden automatisierte Unit-Tests implementiert:

- FSA 3: Korrektheit der Zielfrequenzberechnung anhand bekannter Eingabe- und Ausgabewerte
- FSA 7, FSA 8: Schätzung der Verstimmungsmatrix mittels Deming-Regression, überprüft gegen synthetische Messdaten mit bekannter Steigung

5.5.3 Nicht getestet

Die folgenden Anforderungen wurden im Rahmen dieser Arbeit nicht getestet und stellen offene Punkte für eine Weiterentwicklung dar:

- NFA-LA 1, NFA-LA 2: Latenzmessung erfordert eine instrumentierte Testumgebung zur präzisen Zeiterfassung
- NFA-RO 1: Systematischer Test bei definiertem Umgebungspegel von 70 dB(A) war messtechnisch nicht umsetzbar
- NFA-KO 1 bis NFA-KO 3: Kompatibilitätstests auf verschiedenen Geräten und Betriebssystemversionen stehen aus

6 Evaluation

6.1 Funktionsfähigkeit des Algorithmus

Das zentrale Ziel der Arbeit – eine Floyd-Rose-Gitarre mithilfe der Applikation korrekt zu stimmen – wurde erreicht. In den Nutzertests (Abschnitt 6.3) konnte die Gitarre unter geeigneten akustischen Bedingungen erfolgreich gestimmt werden. Die Verstimmungsmatrix wurde korrekt kalibriert und die berechneten Zielfrequenzen führten zu einem gestimmten Instrument.

Gleichzeitig zeigten die Tests Grenzen des aktuellen Stands: Unter lauten Bedingungen oder bei verzerrtem Signal war die Fundamentalfrequenzerkennung nicht zuverlässig genug, um den Stimmvorgang abzuschließen (Abschnitt 6.3, Nutzer 1). Zudem wurde ein konzeptioneller Fehler im Stimmvorgang identifiziert – die fehlende Kompensation von Folgeverstimmungen beim schrittweisen Stimmen – der zwischenzeitlich behoben wurde (Abschnitt 6.3, Nutzer 2).

6.2 Erfüllung der Requirements aus SWE

ID	Anforderung	Erfüllt
Funktionale Benutzeranforderungen		
FBA 1	Eine Floyd-Rose-Gitarre effizient stimmen	Ja
FBA 2	App auf Gitarre kalibrieren	Ja
FBA 3	Kalibrierung mehrerer Gitarren speichern	Ja
FBA 4	Stimmzustand der Gitarre messen	Ja
FBA 5	Messempfindlichkeit einstellen	Ja
FBA 6	Bedienhilfe erhalten	Teilweise
FBA 7	Korrektheit der Messung manuell prüfen	Ja
FBA 8	Fehlerhafte Messungen korrigieren	Ja
FBA 9	Unterschiedliche Stimmungen auswählen	Ja
FBA 10	Stimmung mit herkömmlichem Stimmgerät prüfen	Ja
FBA 11	App rekalisieren	Ja
FBA 12	Kalibrierungen benennen	Ja
FBA 13	Eigene Stimmungen erstellen, bearbeiten, löschen	Nein
FBA 14	Messungen manuell mit externem Gerät eintragen	Ja
FBA 15	Unbekannte Begriffe nachvollziehen	Nein
FBA 16	Präzision der Kalibrierung erhöhen	Teilweise
Funktionale Systemanforderungen		
FSA 1	Grundfrequenzschätzung per YIN-Algorithmus	Ja
FSA 2	Messung nur bei Überschreitung des dBFS-Schwellenwerts	Ja
FSA 3	Berechnung der Zielfrequenz gemäß Verstimmungsformel	Ja
FSA 4	Stimmen in Reihenfolge $E2 \rightarrow A2 \rightarrow D3 \rightarrow G3 \rightarrow B3 \rightarrow E4$	Ja
FSA 5	Gleitender Mittelwert zur Ausreißerdämpfung	Ja
FSA 6	Frequenzbereich auf 50 Hz – 350 Hz begrenzen	Ja
FSA 7	Verstimmungsmatrix per Deming-Regression schätzen	Ja
FSA 8	Diagonalelemente $C_{ii} = 1$ ohne Messung setzen	Ja
FSA 9	Falsch verstimmte Saite bei Kalibrierung erkennen	Nein
FSA 10	Verstimmungsmatrix C persistent speichern	Ja
FSA 11	Standard-Stimmgerät-Funktionalität anbieten	Ja
FSA 12	Grundfrequenz aus Obertönen ableiten	Nein

ID	Anforderung	Erfüllt
FSA 13	Gitarrenprofilnamen persistent speichern	Ja
FSA 14	Vordefinierte Stimmungen bereitstellen	Ja
FSA 15	Benutzerdefinierte Stimmungen speichern/bearbeiten/löschen	Nein
FSA 16	Mehr als zwei Samples für Regression	Teilweise
Nichtfunktionale Anforderungen – Messgenauigkeit		
NFA-MG 1	Grundfrequenz mit max. ± 0.2 Hz Abweichung	Nein
NFA-MG 2	Obertöne nicht als Grundfrequenz ausgeben	Teilweise
NFA-MG 3	Bei unverstärktem Spiel max. ± 0.1 Hz Abweichung	Nein
Nichtfunktionale Anforderungen – Latenz		
NFA-LA 1	Zielanzeige innerhalb von 100 ms aktualisieren	Nicht Überprüft
NFA-LA 2	Zielfrequenzberechnung in unter 500 ms	Nicht Überprüft
Nichtfunktionale Anforderungen – Robustheit		
NFA-RO 1	Stabile Messung bei bis zu 70 dB(A) Umgebungslärm	Nicht Überprüft
NFA-RO 2	Keine Schätzung bei Stille oder reinem Lärm	Ja
NFA-RO 3	Fallback-Modus bei defektem Mikrofon	Ja
Nichtfunktionale Anforderungen – Bedienbarkeit		
NFA-BE 1	Erststart ohne externe Hilfe abschließbar	Ja
NFA-BE 2	Zentrale Aktionen mit max. 3 Interaktionen erreichbar	Ja
NFA-BE 3	Visuelle statt textbasierte Bedienführung	Teilweise
NFA-BE 4	Erklärvideos bereitstellen	Nein
Nichtfunktionale Anforderungen – Kompatibilität		
NFA-KO 1	Lauffähig auf iOS ≥ 16 und Android ≥ 10	Android: Ja
NFA-KO 2	Betrieb auf Geräten mit ≥ 2 GB RAM	Nicht Überprüft
NFA-KO 3	Kein Layoutbruch auf 4,7 – 6,7 Zoll Displays	Nicht Überprüft
Nichtfunktionale Anforderungen – Datenschutz und Betrieb		
NFA-DP 1	Keine Datenübermittlung an externe Server	Ja
NFA-DP 2	Vollständig offline nutzbar	Ja
NFA-DP 3	Profile und Stimmungen exportierbar/importierbar	Nein

6.3 Nutzertests

6.3.1 Nutzer 0

Die App wurde in einem ruhigen Zimmer mit verstärkter Gitarre ohne Verzerrungseffekt getestet. Die Frequenzen aller Saiten wurden korrekt erkannt und die Gitarre konnte erfolgreich gestimmt werden.

Es traten vereinzelte Schwankungen bei der Erkennung der Fundamentalfrequenz auf, die den Prozess geringfügig verlangsamten.

Die Bestimmung der Verstimmungsmatrix inklusive anschließender Überprüfung der Messdaten dauerte 3:47 Minuten; der gesamte Stimmvorgang war nach ca. 7 Minuten abgeschlossen.

6.3.2 Nutzer 1

Die App wurde auf einer Jam-Session vorgestellt. Beim Versuch, die Frequenzen der E-Gitarre zu messen, wurde nicht die Fundamentalfrequenz, sondern der erste Oberton (Faktor 2) erkannt. Als Ursache kommen zwei Faktoren zusammen: die laute Umgebung sowie ein aktiver Verzerrungseffekt, der den Obertonanteil des Signals verstärkte. Der Stimmvorgang musste abgebrochen werden.

Dieser Test macht deutlich, dass die Fundamentalfrequenzerkennung unter ungünstigen akustischen Bedingungen robuster gestaltet werden muss.

6.3.3 Nutzer 2

Die App wurde zu Hause unter guten akustischen Bedingungen verwendet. Es fiel auf, dass der Kalibrierungsprozess zwar technisch korrekt funktionierte, jedoch unnötige Wiederholungen enthielt: Nachdem der Zustand der Gitarre nach dem Verstimmen einer Saite gemessen wurde, könnte dieser Zustand direkt als Ausgangslage für die Messung der nächsten Saite weiterverwendet werden. Aktuell wird dieser Schritt manuell ausgelöst.

Darüber hinaus zeigte sich ein grundlegendes Problem: Beim Stimmen einer Saite mit Hilfe der App wurden zunächst die korrekten Zielabweichungen Δf angezeigt. Sobald jedoch eine Saite aktiv verstimmt wurde, veränderte sich – erwartungsgemäß aufgrund der Brückenkopplung – der Zustand aller übrigen Saiten. Diese Zustandsänderung wurde nicht in die nachfolgende Berechnung der Δf einbezogen, sodass die Gitarre am Ende nicht präzise gestimmt war. Als Konsequenz muss der Stimmvorgang die durch jede Saitenänderung verursachten Folgeverstimmungen schrittweise vorausberechnen und kompensieren.

6.3.4 Nutzer 3

Nutzer 3 war mit der Darstellung roher Frequenzwerte (in Hz) überfordert und konnte deren Bedeutung nicht einordnen. Dies bestätigte den Bedarf einer abstrahierten, intuitiv verständlichen Visualisierung der Stimmgenauigkeit anstelle numerischer Frequenzangaben.

6.3.5 Nutzer 4

Nutzer 4 versuchte, die Gitarre unmittelbar nach dem Öffnen der App zu stimmen, ohne den Kalibrierungsschritt zu durchlaufen. Die App konnte nicht ausreichend klar vermitteln, dass zunächst mehrere Messreihen zur Bestimmung der Verstimmungsmatrix erforderlich sind. Auch das Tutorial-Video war zu dem Zeitpunkt nicht verfügbar.

Als der Nutzer aufgefordert wurde, eine Saite gezielt zu verstimmen, entstand Unsicherheit darüber, in welche Richtung und um wie viel die Saite verstimmt werden solle – obwohl der genaue Betrag für das Verfahren irrelevant ist.

Zusätzlich hatte die App Schwierigkeiten, die tiefen Saiten korrekt zu messen, da die Stahlsaiten einen ausgeprägten Obertonanteil aufwiesen. Durch Umpositionierung des Smartphones näher am Lautsprecher sowie durch Betätigung des *Tone-Knobs* – der die Funktion eines analogen Tiefpassfilters erfüllt – konnte das Problem behoben werden.

Der abschließende Stimmvorgang verlief weitgehend erfolgreich; die E2-Saite wurde jedoch zu hoch gestimmt. Dies deutet darauf hin, dass die Einträge der Verstimmungsmatrix, die den Einfluss auf E2 beschreiben, den größten Messfehler aufwiesen.

Der effektive Zeitaufwand für den Stimmvorgang betrug 8 Minuten.

6.3.6 Nutzer 5

Mit der finalen UI, dauerte die Kalibrierung etwas länger 7min.

7 Ausblick

7.1 Optimierung der Fundamentalfrequenzschätzung

Die Nutzertests (Abschnitt 6.3) zeigten, dass der YIN-Algorithmus unter zwei Bedingungen zur Erkennung von Obertönen statt der Fundamentalfrequenz neigt: bei hohem Umgebungspegel (Nutzer 1) sowie bei Saiten mit ausgeprägtem Obertonanteil wie tiefen Stahlsaiten (Nutzer 4). In beiden Fällen wurde der erste Oberton – also die doppelte Grundfrequenz – als Schätzwert ausgegeben.

Als Gegenmaßnahme wurde gemäß FSA 12 eine Plausibilitätsprüfung implementiert: Liegt der geschätzte Wert außerhalb des erwarteten Frequenzbereichs der jeweiligen Saite, wird durch Halbierung auf die Grundfrequenz rückgeschlossen. Diese Heuristik behebt den Oktavfehler in den beobachteten Fällen, setzt jedoch voraus, dass der Fehler genau eine Oktave beträgt. Für stärker verrauschte Signale oder höhere Obertöne (Faktor 3, 4, ...) greift sie nicht zuverlässig.

Eine robustere Alternative wäre das *Harmonic Product Spectrum* (HPS): Dabei wird das Frequenzspektrum des Signals mit sich selbst bei schrittweise halbierten Frequenzachsen multipliziert, wodurch die Grundfrequenz als gemeinsamer Teiler aller Partialtöne verstärkt hervortritt. FSA 1 schreibt den YIN-Algorithmus nicht zwingend vor – eine Ersetzung oder Ergänzung durch HPS wäre im Rahmen einer Weiterentwicklung prüfenswert.

7.2 Umsetzung und Testen der verbleibenden Anforderungen

Das Requirements Engineering hat eine Vielzahl klar abgegrenzter Arbeitspakete hervorgebracht. Die verbleibenden Anforderungen zu implementieren, zu verfeinern und durch automatisierte Tests abzusichern würde die Qualität und Zuverlässigkeit der Applikation deutlich verbessern.

7.3 Gleichzeitiges Messen aller Saiten

Derzeit muss der Nutzer jede Saite einzeln anspielen. Durch den Einsatz einer Fourier-Transformation ließe sich das vollständige Frequenzspektrum eines Akkords analysieren, die sechs saitenspezifischen Grundfrequenzen als lokale Maxima identifizieren und harmonische Obertöne herausrechnen. Der Nutzer könnte so alle Saiten gleichzeitig anschlagen, was den Messvorgang erheblich beschleunigen würde.

7.4 Automatische Saitenerkennung beim Stimmen

Wenn der Nutzer während des Stimmvorgangs eine Saite anspielt, könnte die App anhand der gemessenen Frequenz automatisch erkennen, welche Saite verändert wird, und die Anzeige entsprechend umschalten. Voraussetzung ist, dass die Saite bereits näherungsweise auf der Zielfrequenz liegt, sodass eine eindeutige Zuordnung möglich ist.

7.5 Automatisches Fortfahren beim Stimmen

Aktuell muss der Nutzer nach dem Stimmen jeder Saite manuell zur nächsten wechseln. Die App könnte stattdessen automatisch erkennen, wann eine Saite den Zielfrequenzbereich erreicht hat, und ohne Nutzerinteraktion zur nächsten Saite fortfahren. Dies würde den Stimmvorgang weiter beschleunigen und ist besonders für Persona Jonas (Abschnitt 5.1.5.3) relevant, der täglich mehrere Gitarren stimmt.

7.6 Automatische Kalibrierung der Mikrofonempfindlichkeit

Derzeit muss der Nutzer den Empfindlichkeitsschwellenwert manuell über einen Schieberegler einstellen (FBA 5). Die App könnte stattdessen in einer kurzen Einmessphase den Umgebungsgeräuschpegel messen, daraus einen Basiswert ermitteln und den Schwellenwert automatisch so setzen, dass eine gespielte Saite ihn mit ausreichendem Abstand überschreitet. Dies würde FBA 5 obsolet machen und die Einstiegshürde für unerfahrene Nutzer wie Persona Emil weiter senken.

7.7 Mehr Konfigurationsfreiraum

7.7.1 Änderbare Referenzstimmung

Aktuell ist die Referenzfrequenz fest auf 440 Hz eingestellt. Manche Ensembles – insbesondere im Orchesterkontext – stimmen auf abweichende Referenzwerte wie 438 Hz oder 442 Hz. Eine konfigurierbare Referenzfrequenz würde es ermöglichen, die Gitarre präzise auf solche Ensembles abzustimmen.

7.7.2 Unterstützung beliebig vieler Saiten

Die aktuelle Implementierung setzt sechs Saiten voraus. Es gibt jedoch Gitarren mit sieben, acht oder mehr Saiten sowie Bassgitarren mit vier oder fünf Saiten. Eine Verallgemeinerung der Verstimmungsmatrix C auf $n \times n$ wäre konzeptuell *straightforward* und würde die Applikation für eine deutlich breitere Instrumentenvielfalt nutzbar machen.

7.8 Implementierung als DAW-Plugin

Eine Portierung der Kernlogik als VST3- oder CLAP-Plugin in C++ würde den Einsatz in professionellen Studio-Umgebungen ermöglichen und damit das in Abschnitt 5.1.5.4 beschriebene Szenario von Persona Hanna adressieren. Im Plugin-Kontext stünden zudem deutlich präzisere Audio-APIs und geringere Latenzen zur Verfügung, was die Messgenauigkeit weiter steigern könnte. Dieser Anwendungsfall wurde bewusst aus dem Umfang dieser Arbeit ausgeschlossen (siehe Ausgeschlossene Anforderungen), bleibt jedoch ein sinnvoller nächster Schritt.

7.9 Veröffentlichung und Erweiterung auf weitere Brückensysteme

Nach einer abschließenden Qualitätssicherung könnte die Applikation in den App Store und Google Play veröffentlicht werden. Darüber hinaus existieren weitere Gitarrenbrücken – etwa das Kahler-System oder Semi-Floating-Bridges – die ein ähnliches Kopplungsverhalten zwischen den Saiten aufweisen. Eine Verallgemeinerung des Kalibrierungsverfahrens auf solche Systeme würde die potenzielle Nutzerbasis erheblich vergrößern.

Glossar

Sattel Ein fester Punkt am Gitarrenhals (vgl. Abbildung 60).

Brücke Ein fester Punkt am Gitarrenkörper, an dem die Saiten befestigt sind (vgl. Abbildung 60).

Saite Ein dünner Draht, der zwischen Sattel und Brücke einer Gitarre gespannt ist und beim Anschlagen schwingt, um Töne zu erzeugen.

Tremolo Eine spezielle Art von Gitarrenbrücke, die es ermöglicht, die Tonhöhe der Saiten durch Bewegung eines Hebels zu verändern.

Floyd-Rose Erfinder des gleichnamigen Tremolosystems, das in vielen E-Gitarren verwendet wird.

Stimmen einer Gitarre Der Prozess, bei dem die Spannung der Saiten angepasst wird, um die gewünschten Tonhöhen zu erreichen.

Stimmwirbel ein drehbarer Stift aus Metall oder Holz an Saiteninstrumenten, um den das Ende einer Saite gewickelt wird.

E | A | D | G | B | hohe E - Saite Die Namen der sechs Saiten einer Gitarre, von der tiefsten (E) bis zur höchsten (hohe E).

Bund Ein Metallstift, der quer über den Gitarrenhals verläuft und die Saiten in Abschnitte unterteilt, um verschiedene Töne zu erzeugen, wenn die Saite auf den Bund gedrückt wird.

MAUI Multi-platform App UI

Stimmung Die Stimmung im Kontext von Gitarren bezeichnet die spezifische Tonhöhe, auf die die sechs Saiten des Instruments eingestellt sind. Sie legt fest, welche Töne erklingen, wenn die Saiten leer (ohne Greifen im Bund) angeschlagen werden.

Tuning der englische Begriff für Stimmung

DAW Digital Audio Workstation

VST Virtual Studio Technology ist eine Programmierschnittstelle für Audio-Plug-ins. Damit können virtuelle Effekte programmiert werden.

CLAP CLever Audio Plug-in

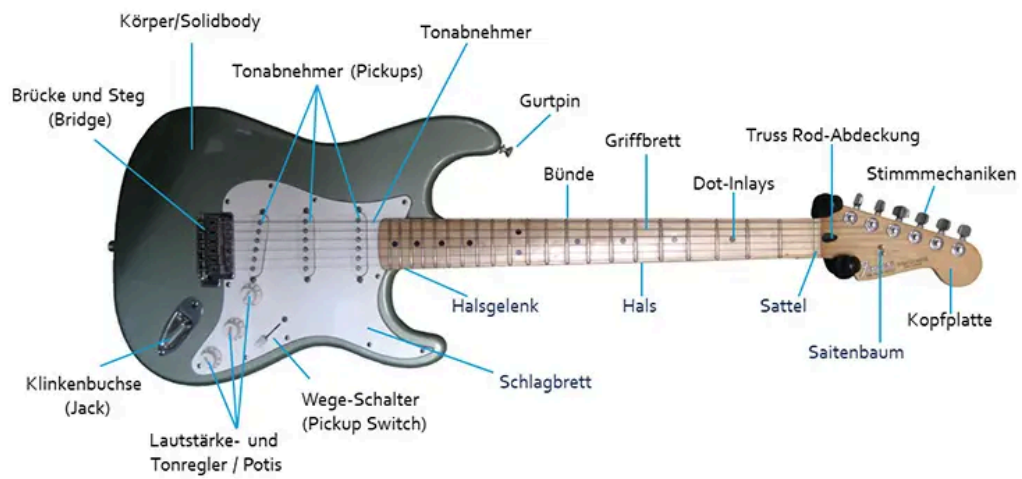


Abbildung 60: Begriffe einer Gitarre

Abbildungsverzeichnis

Abbildung 1	Floyd-Rose-Tremolo Bild	5
Abbildung 2	Gitarre Stimmwirbel	5
Abbildung 3	Bund 1 - Ruhe	6
Abbildung 4	Bund 1 - Spannung	6
Abbildung 5	Bund 2 - Ruhe	6
Abbildung 6	Bund 2 - Spannung	6
Abbildung 7	Bund 4 - Ruhe	6
Abbildung 8	Bund 4 - Spannung	6
Abbildung 9	Bund 6 - Ruhe	6
Abbildung 10	Bund 6 - Spannung	6
Abbildung 11	Bund 8 - Ruhe	6
Abbildung 12	Bund 8 - Spannung	6
Abbildung 13	Bund 12 - Ruhe	7
Abbildung 14	Bund 12 - Spannung	7
Abbildung 15	Bund 16 - Ruhe	7
Abbildung 16	Bund 16 - Spannung	7
Abbildung 17	Bund 22 - Ruhe	7
Abbildung 18	Bund 22 - Spannung	7
Abbildung 19	Floyd-Rose-Modell Quer	8
Abbildung 20	Tremolofedern	9
Abbildung 21	Floyd-Rose-Modell Draufsicht	9
Abbildung 22	Floyd-Rose Draufsicht	9
Abbildung 23	Relativer Einfluss der E2 Saite auf die anderen Saiten	13
Abbildung 24	Relativer Einfluss der A2 Saite auf die anderen Saiten	13
Abbildung 25	Relativer Einfluss der D3 Saite auf die anderen Saiten	13
Abbildung 26	Relativer Einfluss der G3 Saite auf die anderen Saiten	13
Abbildung 27	Relativer Einfluss der B3 Saite auf die anderen Saiten	13
Abbildung 28	Relativer Einfluss der E4 Saite auf die anderen Saiten	13
Abbildung 29	Pearson-Korrelationskoeffizienten der Messdaten	14
Abbildung 30	Verstimmungsmatrix Beispiel	15
Abbildung 31	Informationsarchitektur der Floyd-Rose-Tuner-App	29
Abbildung 32	Interaktionsdesign – Navigation	30
Abbildung 33	Interaktionsdesign – Kalibrierung	30
Abbildung 34	Interaktionsdesign – Stimmprozess	31
Abbildung 35	Früher Prototyp: Messung der Verstimmungsmatrix	32
Abbildung 36	Überarbeiteter Prototyp: Kalibrierungsansicht	32
Abbildung 37	Wireframe: Startseite	32
Abbildung 38	Wireframe: Gitarre bearbeiten	32
Abbildung 39	Wireframe: Kalibrierung – Saite messen	32
Abbildung 40	Wireframe: Kalibrierung – Messung prüfen	33
Abbildung 41	Wireframe: Kalibrierung – Saite verstimmen	33
Abbildung 42	Wireframe: Kalibrierung – Gesamtübersicht	33
Abbildung 43	Wireframe: Hilfsseite	33
Abbildung 44	Wireframe: Standard-Stimmgerät	33
Abbildung 45	Design: Hilfsseite	34
Abbildung 46	Design: Startseite	34
Abbildung 47	Design: Gitarre bearbeiten	34

Abbildung 48	Design: Kalibrierung – Saite messen	35
Abbildung 49	Design: Manuelle Frequenzeingabe	35
Abbildung 50	Design: Kalibrierung – Messung prüfen	35
Abbildung 51	Design: Kalibrierung – Saite verstimmen	35
Abbildung 52	Design: Kalibrierung – Gesamtübersicht	35
Abbildung 53	Design: Gitarre auswählen	35
Abbildung 54	Design: Rekalibrieren	36
Abbildung 55	Design: Floyd-Rose-Tuner	36
Abbildung 56	Design: Standard-Stimmgerät	36
Abbildung 57	Architektur der App	37
Abbildung 58	Zustandsdiagramm: Kalibrierungslogik	42
Abbildung 59	Zustandsdiagramm: Stimmlogik	44
Abbildung 60	Begriffe einer Gitarre	53

Tabellenverzeichnis

Tabelle 1	Elastizität von Gitarrensaiten	6
Tabelle 2	Saitennamen mit Frequenzen	12

Bibliografie

- [1] YouTube, „Quickly Tune A Floyd Rose Trem“. [Online]. Verfügbar unter: <https://youtu.be/z8ftAOFr4sg?t=239>
- [2] U. G. Forum, „How long does it take you to tune your Floyd Rose guitar?“. [Online]. Verfügbar unter: <https://www.ultimate-guitar.com/forum/showthread.php?t=1125303>
- [3] R. Schütz, „Anhänge zur Bachelorarbeit: Floyd-Rose-Tuner App“. Zugegriffen: 18. April 2026. [Online]. Verfügbar unter: https://github.com/Raphael2b3/bachelorthesis_floyd_rose_tuner
- [4] J. Jeans, *Science & Music*. in (Dover books. no. T 1964.). Dover Publications, 1968. [Online]. Verfügbar unter: <https://books.google.de/books?id=dF7FVc9Tgz4C>
- [5] F. P. Beer, E. R. Johnston, J. T. DeWolf, und D. F. Mazurek, *Mechanics of Materials*, 8th Aufl. New York: McGraw-Hill Education, 2020.
- [6] P. A. Tipler und G. Mosca, *Physik für Studierende der Natur- und Ingenieurwissenschaften*, 7. Auflage. Berlin: Springer Spektrum, 2015.
- [7] P. Glaister, „85.13 Least squares revisited“, *The Mathematical Gazette*, Bd. 85, Nr. 502, S. 104–107, 2001, doi: 10.2307/3620485.
- [8] K. Pearson, „Mathematical Contributions to the Theory of Evolution. III. Regression, Heredity, and Panmixia“, *Philosophical Transactions of the Royal Society of London. Series A*, Bd. 187, S. 253–318, 1896, doi: 10.1098/rsta.1896.0007.
- [9] A. Cheveigné und H. Kawahara, „YIN, A fundamental frequency estimator for speech and music“, *The Journal of the Acoustical Society of America*, Bd. 111, S. 1917–1930, 2002, doi: 10.1121/1.1458024.
- [10] M. Stanek und T. Smatana, „Comparison of fundamental frequency detection methods and introducing simple self-repairing algorithm for musical applications“, in *2015 25th International Conference Radioelektronika (RADIOELEKTRONIKA)*, 2015, S. 217–221. doi: 10.1109/RADIO-ELEK.2015.7129013.
- [11] A. M. Noll, „Cepstrum Pitch Determination“, *The Journal of the Acoustical Society of America*, Bd. 41, Nr. 2, S. 293–309, 1967, doi: 10.1121/1.1910400.
- [12] J. W. Kim und J. P. Bello, „CREPE: A Convolutional Representation for Pitch Estimation“, in *Proceedings of the International Society for Music Information Retrieval Conference*, 2019. [Online]. Verfügbar unter: <https://arxiv.org/abs/1802.06291>
- [13] A. V. Oppenheim, R. W. Schaffer, und J. R. Buck, *Discrete-Time Signal Processing*, 2. Aufl. Upper Saddle River, NJ: Prentice Hall, 1999.
- [14] J. Watkinson, *The Art of Digital Audio*, 3. Aufl. Focal Press, 2001.
- [15] G. Vollmer, *Mobile App Engineering : eine systematische Einführung - von den Requirements zum Go Live /*, 1. Auflage. Heidelberg: : dpunkt.verlag, 2017. [Online]. Verfügbar unter: <http://vub.de/cover/data/isbn:9783864904219/medium/true/de/vub/cover.jpg>
- [16] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA: Addison-Wesley Professional, 2004.
- [17] J. J. Garrett, *Die Elemente der User Experience: Anwenderzentriertes (Web-)Design*. München: Pearson Deutschland, 2012.

- [18] Google, „Material Design 3“. Zugegriffen: 21. Mai 2026. [Online]. Verfügbar unter: <https://m3.material.io/>
- [19] Meta Platforms, Inc., „React Native: Learn once, write anywhere“. [Online]. Verfügbar unter: <https://reactnative.dev/>
- [20] Microsoft, „NET Multi-platform App UI (.NET MAUI)“. [Online]. Verfügbar unter: <https://dotnet.microsoft.com/en-us/apps/maui>
- [21] Google, „Flutter: Build apps for any screen“. [Online]. Verfügbar unter: <https://flutter.dev/>